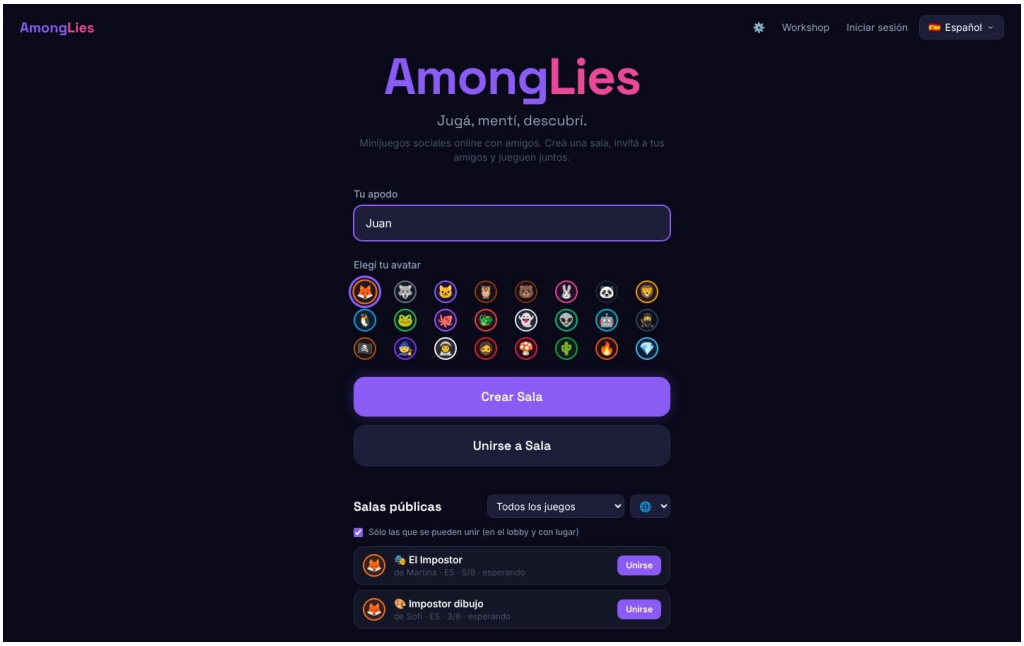
AmongLies

Jugá, mentí, descubrí.

Documentación de Arquitectura

Versión final del sistema



Trabajo Práctico N.° 3 — Documentación

Materia: Desarrollo de Sistemas

Integrantes: Juan Cruz Palacios, Matias Budani y Valentino Barrio

Octubre de 2026

Índice

1. Introducción y objetivos 5
1.1 Propósito del sistema 5
1.2 Objetivos del proyecto 5
1.3 Requerimientos funcionales principales 5
1.4 Objetivos de calidad 6
1.5 Partes interesadas 7
2. Restricciones 8
2.1 Restricciones técnicas 8
2.2 Restricciones organizativas 8
2.3 Convenciones 8
2.4 Restricciones legales y éticas 8
3. Contexto y alcance 10
3.1 Contexto de negocio (C4 — Nivel 1) 10
3.2 Contexto técnico 10
3.3 Alcance 11
4. Estrategia de solución 12
4.1 Estilo arquitectónico 12
4.2 Enfoques por objetivo 12
4.3 De la tecnología al impacto para el usuario 12
5. Vista de bloques de construcción 14
5.1 Nivel 1 — Contenedores (C4 — Nivel 2) 14
5.2 Nivel 2 — Componentes (C4 — Nivel 3) 15
5.3 Nivel 3 — Código (C4 — Nivel 4): el contrato de los motores de juego 16
5.4 Estructura del repositorio 17
6. Vista de ejecución 18
6.1 Escenario 1 — Crear una sala y unirse 18
6.2 Escenario 2 — Una ronda de El Impostor 19
6.3 Máquina de estados del motor El Impostor 20
6.4 Escenario 3 — Fin del juego y registro de estadísticas 21
6.5 Escenario 4 — Inicio de sesión y conexión autenticada 21
6.6 Escenario 5 — Desconexión y reconexión 22
6.7 Catálogo de eventos 22
7. Vista de despliegue 24
Entornos 25
8. Conceptos transversales 26
8.1 Modelo de dominio 26
8.2 Diseño de datos 26
8.3 Contrato de comunicación 28
8.4 Seguridad e integridad del juego 28

8.5 Manejo del tiempo....29
8.6 Identidad y sesiones....29
8.7 Internacionalización....29
8.8 Manejo de errores y retroalimentación....29
8.9 Escalabilidad....29
8.10 Observabilidad....30
8.11 Estrategia de pruebas....30
8.12 Diseño de la interfaz: decisiones y justificación....30
9. Decisiones de arquitectura....33
9.1 Criterios de evaluación....33
ADR-01 — Comunicación en tiempo real con Socket.io....33
ADR-02 — Servidor autoritativo....34
ADR-03 — Monolito modular con NestJS....34
ADR-04 — Next.js (App Router) para la aplicación web....34
ADR-05 — Estado vivo en memoria (sin Redis)....34
ADR-06 — PostgreSQL (Supabase) para datos persistentes....35
ADR-07 — Cuentas opcionales (invitado primero) con Supabase Auth....35
ADR-08 — Motores de juego enchufables....35
ADR-09 — Vistas por jugador calculadas en el servidor....36
ADR-10 — Modo voz sin audio propio....36
ADR-11 — Zustand para el estado del cliente....36
ADR-12 — Monorepo con tipos compartidos....36
ADR-13 — Despliegue en Render....37
ADR-14 — Listado de salas públicas por REST....37
ADR-15 — Resultados escritos sólo por el servidor....37
ADR-16 — Amigos y workshop por el socket del juego....37
ADR-17 — Moderación: filtro compartido, reportes y panel de admin....38
ADR-18 — Internacionalización propia en lugar de una librería....38
ADR-19 — Sonido sintetizado con Web Audio....38
ADR-20 — Identidad de invitado con un token por pestaña....38
ADR-21 — Cuentas con email y contraseña (sin login con Google)....39
ADR-22 — Pruebas de punta a punta con navegador real y bots....39
ADR-23 — Mantener despierto el servidor gratuito en lugar de pagar....39
10. Requerimientos de calidad....40
10.1 Árbol de calidad....40
10.2 Escenarios de calidad....40
10.3 Trazabilidad: requerimiento → implementación → prueba....41
11. Riesgos y deuda técnica....43
11.1 Riesgos....43
11.2 Deuda técnica....43
12. Glosario....44

Anexo A — Referencias..... 46

1. Introducción y objetivos

1.1 Propósito del sistema

AmongLies es una **plataforma web de minijuegos sociales multijugador en tiempo real**, pensada para que un grupo de amigos pueda juntarse a jugar desde el navegador —en la computadora o en el celular— sin instalar nada y sin la obligación de registrarse. Se inspira en juegos como *Among Us*, *Spyfall* y *Gartic Phone*: juegos cortos de deducción, donde lo divertido es la interacción entre las personas. Los tres juegos de la plataforma comparten la misma idea: hay uno o más **impostores** escondidos y el grupo tiene que descubrirlos votando.

El problema que resuelve es concreto: organizar una partida de este tipo de juegos entre amigos suele requerir papel, un moderador que no juega, o aplicaciones que obligan a todos a crear una cuenta antes de empezar. AmongLies automatiza el rol de moderador (reparte roles secretos, controla turnos y tiempos, cuenta votos) y reduce la entrada a **elegir un apodo y compartir un código**.

1.2 Objetivos del proyecto

Los objetivos están formulados de forma verificable, para poder evaluar si la arquitectura los cumple. La última columna indica cómo se verificó cada uno en el sistema construido:

ID	Objetivo	Cómo se mide / verificó
O1	**Jugar sin fricción**: cualquier persona puede pasar de abrir el link a estar dentro de una sala en menos de 30 segundos, sin registrarse.	Escenario QS-5. Se entra con apodo, avatar y código o link; la cuenta es opcional.
O2	**Experiencia en tiempo real**: todos los jugadores de una sala ven el mismo estado del juego prácticamente al mismo tiempo.	QS-1. Partidas de 4 navegadores contra producción; el dibujo se transmite trazo a trazo.
O3	**Juego justo**: ningún jugador puede ver información que no le corresponde ni hacer trampa manipulando el cliente.	QS-3 y QS-4. Tests unitarios ("el impostor nunca recibe la palabra") y suite e2e `servidor`.
O4	**Catálogo ampliable**: tres minijuegos y la posibilidad de sumar otros sin reescribir salas, chat ni conexión.	QS-7. Impostor por tiempo e Impostor dibujo se agregaron sin tocar salas, chat ni transporte.
O5	**Progreso persistente opcional**: quien cree una cuenta conserva perfil, estadísticas, amigos y su colección del workshop.	RF-14 a RF-16 y RF-18 a RF-20 implementados y probados de punta a punta con cuentas temporales.
O6	**Accesible y multi-idioma**: interfaz usable desde 360 px de ancho, en español e inglés.	QS-6. Pruebas e2e en un celular táctil; ninguna página tiene scroll horizontal a 360 px.
O7	**Costo operativo mínimo**: operar dentro de planes gratuitos durante la etapa académica.	Render (plan gratis) + Supabase (plan gratis) + GitHub Actions: USD 0 por mes.

Tabla 1 — Objetivos del proyecto

1.3 Requerimientos funcionales principales

Todos los requerimientos están **implementados** y probados en la versión final.

ID	Requerimiento	Actor	Estado
RF-01	Ingresar con apodo (2–16 caracteres) y avatar (24 opciones), recordados en el navegador.	Todos	Hecho
RF-02	Crear una sala con código único de 6 caracteres y link para compartir; privada por defecto.	Todos	Hecho
RF-03	Unirse a una sala por código, abriendo el link directo o desde una invitación.	Todos	Hecho

ID	Requerimiento	Actor	Estado
RF-04	Chat de sala en tiempo real (lobby y juego), con panel flotante en el celular y censura de insultos.	Todos	Hecho
RF-05	Elegir minijuego y configurar sus reglas (partidas, impostores, tiempos, voto secreto, desempate, listas de palabras, modo chat/voz…).	Anfitrión	Hecho
RF-06	Expulsar jugadores y transferir el rol de anfitrión; si el anfitrión se desconecta, decide el jugador conectado más antiguo.	Anfitrión	Hecho
RF-07	Jugar **El Impostor**: roles secretos, pistas por turnos, discusión, votación, resultados, partidas y rondas, puntaje.	Jugadores	Hecho
RF-08	Modo voz: la app controla turnos y tiempos; la conversación ocurre fuera (presencial o por llamada).	Jugadores	Hecho
RF-09	Interfaz en español e inglés; listas de palabras según el idioma de la sala.	Todos	Hecho
RF-10	Jugar **Impostor por tiempo**: parar un reloj oculto lo más cerca posible de un tiempo que el impostor no conoce.	Jugadores	Hecho
RF-11	Jugar **Impostor dibujo**: todos dibujan la palabra secreta en un lienzo compartido con tinta limitada; el impostor no la conoce.	Jugadores	Hecho
RF-12	Marcar una sala como **pública**, definir cupo e idioma, y listarla en la página de inicio.	Anfitrión	Hecho
RF-13	Ver salas públicas filtrando por juego e idioma, y unirse con un clic.	Todos	Hecho
RF-14	Registrarse e iniciar sesión con email y contraseña; recuperar la contraseña. La cuenta es **opcional**.	Invitado	Hecho
RF-15	Perfil con nombre de usuario único y avatar.	Registrado	Hecho
RF-16	Estadísticas por juego y por rol (partidas, victorias como impostor e inocente, votos acertados, puntos).	Registrado	Hecho
RF-17	Reconexión: si se corta la conexión o se recarga la página, volver a la misma sala y partida; la partida se pausa mientras tanto.	Todos	Hecho
RF-18	**Amigos**: solicitudes, presencia en vivo (en línea, en una sala, jugando) e invitaciones a la sala.	Registrado	Hecho
RF-19	**Workshop**: crear, publicar, buscar, dar like y guardar listas de palabras y presets de reglas; usarlos en el lobby.	Registrado	Hecho
RF-20	**Moderación**: reportar jugadores y contenido; panel de admin para resolver reportes, ocultar contenido y suspender cuentas.	Todos / Admin	Hecho
RF-21	**Ajustes**: idioma, efectos y música, avisos, privacidad, cambio de email y contraseña, borrar la cuenta.	Todos	Hecho

Tabla 2 — Requerimientos funcionales y estado de implementación

1.4 Objetivos de calidad

De todos los atributos de calidad posibles, el equipo priorizó cinco. El orden importa: cuando dos objetivos entran en conflicto, gana el de mayor prioridad. Cada uno se detalla con escenarios medibles en la sección 10.

Prioridad	Atributo	Motivación
1	**Rendimiento en tiempo real**	Un juego social se arruina si un jugador ve el turno o el voto con segundos de retraso respecto de los demás.
2	**Integridad del juego** (seguridad)	Todo el juego depende de información oculta. Si el impostor puede leer la palabra desde las herramientas del navegador, el juego deja de tener sentido.
3	**Usabilidad / baja fricción**	El público es casual. Cada paso extra antes de jugar (registro, instalación) reduce cuántas personas llegan a jugar.
4	**Extensibilidad**	El producto es una plataforma de varios minijuegos; agregar uno nuevo debe ser barato.
5	**Disponibilidad y escalabilidad**	Soportar el crecimiento de salas simultáneas y reinicios del servidor sin cortar partidas.

Tabla 3 — Objetivos de calidad priorizados

1.5 Partes interesadas

Rol	Expectativa sobre el sistema	Qué busca en este documento
Jugador casual	Entrar rápido, que funcione en el celular, que sea justo y divertido.	Manual de usuario: cómo se juega.
Anfitrión de sala	Controlar la sala: configurar, moderar, invitar.	Manual de usuario: permisos y configuración.
Usuario registrado	Conservar su progreso, sus amigos y lo que crea en el workshop.	Manual de usuario: perfil, amigos y workshop.
Administrador	Mantener la comunidad sana: revisar reportes y suspender a quien abusa.	Manual de usuario: moderación (M14).
Equipo de desarrollo	Entender la estructura para mantenerla y extenderla; incorporar integrantes.	Secciones 4 a 9 y 11.
Docente / evaluador	Verificar que las decisiones estén justificadas con criterios.	Secciones 1, 3, 4, 9 y 10.
Operador del despliegue	Saber qué se despliega, dónde y con qué configuración.	Sección 7.

Tabla 4 — Partes interesadas

2. Restricciones

Las restricciones son condiciones que el equipo no eligió libremente y que limitan el espacio de soluciones. Muchas de las decisiones de la sección 9 se explican por ellas.

2.1 Restricciones técnicas

ID	Restricción	Consecuencia en la arquitectura
RT-1	Debe funcionar en el **navegador**, en desktop y móvil, sin instalar aplicaciones.	Aplicación web responsive; nada de clientes nativos ni plugins.
RT-2	La comunicación en tiempo real requiere **conexiones persistentes** (WebSockets).	El servidor de juego debe ser un proceso de larga duración: no puede desplegarse en plataformas serverless como Vercel Functions.
RT-3	**TypeScript** en todo el stack (conocimiento del equipo).	Node.js en el servidor; tipos compartidos entre cliente y servidor.
RT-4	Infraestructura dentro de **planes gratuitos**.	Servicios gestionados (Render, Supabase). En el plan gratis de Render el servidor se duerme tras 15 minutos sin tráfico: la aplicación lo tiene en cuenta (§8.8).
RT-5	Sin servidor de audio/video propio.	El modo voz sólo coordina turnos; la conversación ocurre en otra app o en persona.

Tabla 5 — Restricciones técnicas

2.2 Restricciones organizativas

ID	Restricción	Consecuencia
RO-1	Equipo reducido de estudiantes con dedicación parcial.	Preferir frameworks con convenciones fuertes (NestJS, Next.js) y un único repositorio.
RO-2	Plazos fijados por el calendario de la materia; entregas incrementales.	Arquitectura que crezca sin reescrituras: motores enchufables y tipos compartidos desde el principio. La versión final se construyó en 9 fases, cada una verificada antes de la siguiente.
RO-3	Control de versiones en GitHub; nada se integra a `main` sin pasar los controles.	Hook pre-commit (secretos, lint, tests) y CI en cada push; despliegue automático desde `main`.

Tabla 6 — Restricciones organizativas

2.3 Convenciones

- Idioma de la interfaz por defecto: español rioplatense; código y nombres técnicos en inglés.
- Eventos de tiempo real nombrados `dominio:acción` (por ejemplo `room:join`, `game:action`), definidos una sola vez en el paquete compartido.
- Estilo visual “Neon Noir” (fondo azul muy oscuro, violeta como color primario, rosa para el impostor) definido como variables de diseño.

2.4 Restricciones legales y éticas

- **Ley 25.326 de Protección de Datos Personales (Argentina)**: se minimizan los datos recolectados. Los invitados no dejan datos personales; los usuarios registrados sólo entregan un email (gestionado por Supabase Auth) y un nombre de usuario público. Cada usuario puede **borrar su cuenta** y todo lo asociado desde Ajustes.

- El chat no se guarda: vive en memoria mientras exista la sala. La única excepción es un **reporte**: se adjuntan como evidencia los últimos 20 mensajes de la sala, que sólo ven los administradores.
- Contenido apto para todo público: las listas de palabras propias son revisadas por el filtro de insultos, y el contenido del workshop se puede reportar y ocultar.

3. Contexto y alcance

3.1 Contexto de negocio (C4 — Nivel 1)

El diagrama de contexto muestra el sistema como una caja negra: quiénes lo usan y con qué sistemas externos se relaciona. Es la vista pensada para personas no técnicas.

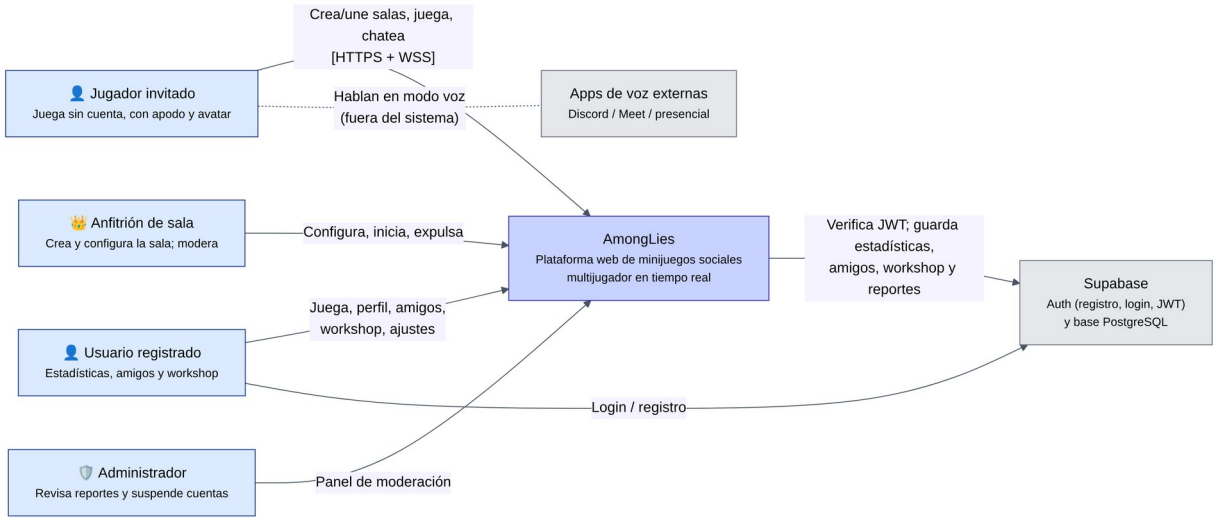


Figura 1 — C4 Nivel 1: contexto del sistema AmongLies

Actor / sistema	Tipo	Qué intercambia con AmongLies
Jugador invitado	Persona	Apodo, avatar, código de sala, mensajes de chat, pistas, trazos, votos. Recibe el estado del juego.
Usuario registrado	Persona	Lo mismo que el invitado, más credenciales; recibe perfil, estadísticas, amigos y su colección del workshop.
Anfitrión de sala	Rol de persona	Configuración de la partida, órdenes de iniciar, expulsar y transferir el rol.
Administrador	Rol de persona	Revisa reportes, oculta contenido del workshop y suspende cuentas desde el panel de admin.
Supabase	Sistema externo	Auth: registro, login, recuperación de contraseña y emisión de JWT. PostgreSQL: perfiles, estadísticas, amistades, workshop y reportes, con RLS.
Apps de voz externas	Fuera del sistema	En modo voz los jugadores hablan por Discord, Meet o en persona; AmongLies no transmite audio.

Tabla 7 — Interfaces del contexto de negocio

3.2 Contexto técnico

Canal	Protocolo	Formato	Uso
Navegador → Aplicación Web	HTTPS	HTML/JS	Carga de la aplicación.
Navegador ↔ Game Server	Socket.io (HTTP long-polling que sube a WebSocket)	Eventos JSON tipados	Salas, chat, juego, amigos, workshop y moderación.
Navegador → Game Server	HTTPS REST	JSON	`GET /rooms/public` (salas públicas) y `GET /health` (mantener despierto el servidor).

Canal	Protocolo	Formato	Uso
Navegador → Supabase	HTTPS	JSON (supabase-js)	Login y registro; lectura de perfiles, estadísticas y workshop (protegida por RLS).
Game Server → Supabase Auth	HTTPS	JWKS	Claves públicas para verificar los JWT de las cuentas.
Game Server → Supabase PostgreSQL	HTTPS (API REST)	JSON	Escritura de estadísticas, amistades, workshop y reportes con la clave secreta del servidor.

Tabla 8 — Contexto técnico: canales y protocolos

3.3 Alcance

Definir qué queda **fuera** es tan importante como definir qué entra: evita que el proyecto crezca sin control y explica por qué ciertas funciones no existen.

Dentro del alcance (versión final)	Fuera del alcance
Salas privadas (código, link, invitación) y públicas (listado en la página de inicio) con cupo configurable.	Matchmaking automático con desconocidos por nivel o ranking.
Tres minijuegos de deducción: El Impostor, Impostor por tiempo e Impostor dibujo.	Juegos de otro tipo (por ejemplo, de dibujo y adivinanza sin impostor).
Cuentas opcionales, perfil y estadísticas por juego.	Historial detallado partida por partida; rankings globales.
Amigos con presencia e invitaciones a salas.	Mensajes privados entre usuarios; notificaciones push.
Workshop de listas de palabras y presets de reglas hechos por los usuarios.	Editor de minijuegos o de estética (temas, avatares propios).
Moderación: filtro de insultos, reportes, panel de admin y suspensiones.	Moderación automática con inteligencia artificial.
Chat de sala en tiempo real; sonido sintetizado en el navegador.	Chat de voz o video integrado (se usa una app externa).
Español e inglés. Aplicación web responsive.	Otros idiomas; aplicaciones nativas; monetización.

Tabla 9 — Alcance de la versión final

4. Estrategia de solución

4.1 Estilo arquitectónico

AmongLies combina tres estilos, cada uno elegido por un objetivo concreto:

- **Cliente–servidor con servidor autoritativo.** El servidor es la única fuente de verdad del estado del juego. Los clientes nunca modifican el estado: sólo envían *intenciones* ("quiero votar a Sofi"), que el servidor valida contra las reglas antes de aplicarlas. Esto responde al objetivo de integridad (O3).
- **Arquitectura orientada a eventos.** Cliente y servidor se comunican con eventos con nombre sobre una conexión persistente. El servidor puede *empujar* cambios a todos los jugadores de una sala en el instante en que ocurren. Responde al objetivo de tiempo real (O2).
- **Monolito modular.** El servidor es una sola aplicación desplegable, dividida en módulos por dominio (salas, chat, juego, jugadores, estadísticas, amigos, workshop, moderación) con límites claros. Para un equipo chico da la organización de los microservicios sin su costo operativo (ADR-03).

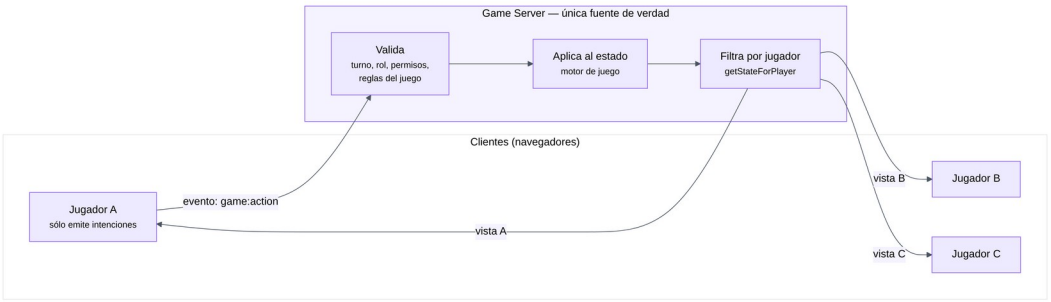


Figura 2 — Servidor autoritativo: los clientes emiten intenciones y reciben vistas filtradas

4.2 Enfoques por objetivo

Objetivo	Enfoque de solución	Detalle
O1 Sin fricción	Identidad de invitado por defecto; cuenta opcional. Salas por código de 6 caracteres sin letras ambiguas, link e invitaciones de amigos.	§8.6, ADR-07
O2 Tiempo real	Socket.io; difusión por sala; estado vivo en memoria, nunca en la base relacional; trazos del dibujo como eventos livianos.	ADR-01, ADR-05
O3 Juego justo	Servidor autoritativo; vista por jugador (`getStateForPlayer`); validación de turno, rol, permisos y tinta en el servidor.	§8.4, ADR-02, ADR-09
O4 Catálogo ampliable	Interfaz `GameEngine` + registro de juegos; `DeductionEngine` con lo común de la familia Impostor.	§5.3, ADR-08
O5 Progreso persistente	Supabase (Auth + PostgreSQL) con Row Level Security; todo lo que cuenta lo escribe sólo el servidor.	§8.2, ADR-06, ADR-15
O6 Accesible y multi-idioma	Diseño mobile-first con Tailwind; diccionarios es/en y listas de palabras por idioma.	§8.7, §8.12
O7 Costo mínimo	Planes gratuitos de Render y Supabase; un monolito en lugar de muchos servicios.	§7, ADR-13

Tabla 10 — Estrategia de solución por objetivo

4.3 De la tecnología al impacto para el usuario

Cada elección técnica tiene sentido sólo si produce un beneficio observable. La siguiente tabla traduce las decisiones principales a lo que percibe quien juega:

Decisión técnica	Impacto para el usuario
WebSockets con difusión por sala.	Cuando alguien vota o dibuja, todos lo ven al instante; no hay que recargar la página.
Servidor autoritativo con vistas filtradas por jugador.	Nadie puede descubrir al impostor ni la palabra "espiando" el navegador: el dato nunca llega a su dispositivo.
Identidad de invitado + código de sala.	Entrar a jugar toma segundos: apodo, avatar y código.
Reconexión con pausa de la partida.	Si a alguien se le corta internet o recarga la página, vuelve a su lugar y la partida lo espera.
Motores de juego enchufables con una base común.	Los tres juegos se juegan igual (roles, discusión, votación, puntaje): aprender uno es aprender los tres.
Supabase Auth + RLS.	Tus estadísticas, tu colección y tus amigos son tuyos; nadie puede modificar lo de otro.

Tabla 11 — Traducción de decisiones técnicas a impacto

5. Vista de bloques de construcción

Esta sección descompone el sistema de lo general a lo particular. Cada nivel “abre” una caja del nivel anterior.

5.1 Nivel 1 — Contenedores (C4 — Nivel 2)

Un contenedor es una unidad que se ejecuta o almacena datos por separado. AmongLies tiene tres contenedores propios y un paquete compartido que no se ejecuta solo, sino que se compila dentro de los otros.

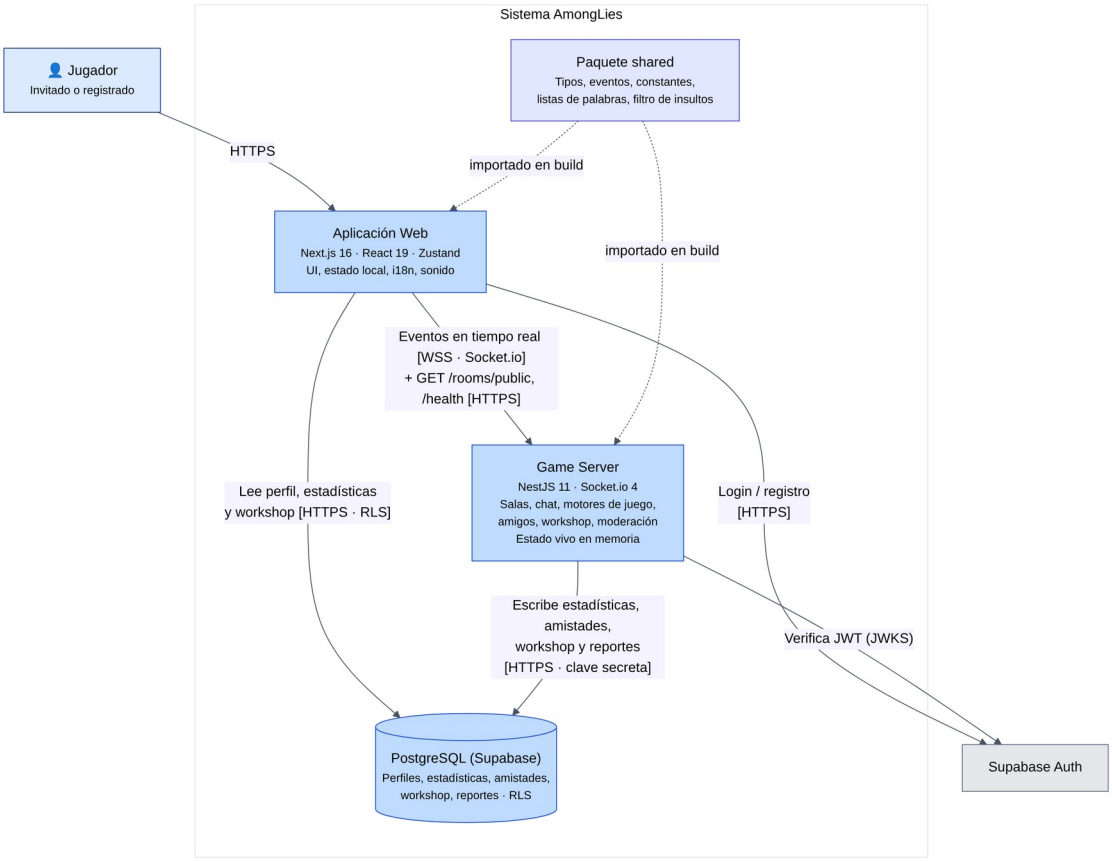


Figura 3 — C4 Nivel 2: contenedores de AmongLies

Contenedor	Tecnología	Responsabilidad	Por qué existe separado
Aplicación Web	Next.js 16 (App Router), React 19, Tailwind 4, Framer Motion, Zustand	Interfaz, animaciones, estado local, idioma, sonido sintetizado, conexión con el servidor y lecturas de Supabase.	Ciclo de despliegue distinto del servidor de juego; sirve contenido estático cacheable.
Game Server	NestJS 11, Socket.io 4, Node 22	Salas, chat, sesiones y reconexión, motores de juego y temporizadores, amigos, workshop, moderación, salas públicas, registro de estadísticas. **El estado vivo está en su memoria.**	Proceso de larga duración que mantiene conexiones persistentes (RT-2).
PostgreSQL + Auth	Supabase (PostgreSQL 17 + Auth)	Cuentas, perfiles, estadísticas, amistades, workshop y reportes.	Datos que deben sobrevivir y consultarse con relaciones y agregaciones.
Paquete shared	TypeScript	Tipos de dominio, contrato de eventos, definiciones de juegos, puntaje, listas de palabras, filtro de insultos y cálculo de tinta.	Una única fuente de verdad del contrato cliente–servidor y de la lógica que corren los dos lados.

Tabla 12 — Contenedores y responsabilidades

5.2 Nivel 2 — Componentes (C4 — Nivel 3)

Componentes del Game Server

El servidor se organiza en módulos de NestJS. La **capa de transporte** (gateways y controladores) traduce eventos de red en llamadas y verifica permisos; la **capa de dominio** contiene las reglas. Las reglas del juego nunca dependen de Socket.io, lo que permite probarlas de forma aislada con timers simulados.

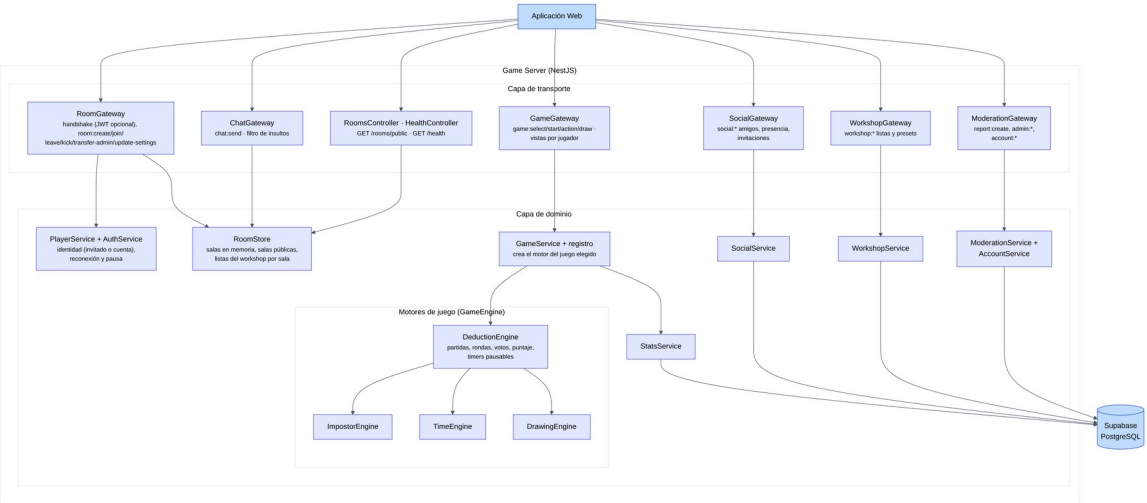


Figura 4 — C4 Nivel 3: componentes del Game Server

Componente	Responsabilidad
RoomGateway	Conexión: identidad de invitado (sessionToken) o de cuenta (JWT verificado con el JWKS de Supabase), reconexión y pausa. Eventos de sala: crear, unirse, salir, expulsar, transferir anfitrión, cambiar ajustes de la sala. Rechaza apodos con insultos y cuentas suspendidas.
ChatGateway	Recibe mensajes, recorta a 200 caracteres, censura insultos con `***` y los difunde a la sala. Rechaza mensajes de eliminados durante la partida.
GameGateway + GameService	Selección de juego, ajustes, inicio, acciones y trazos del dibujo; crea el motor del juego elegido y **difunde una vista distinta a cada jugador**. Resuelve las listas de palabras (del juego o del workshop).
Motores de juego	`DeductionEngine` (partidas, rondas, turnos, votación, desempates, puntaje, pausa) y sus tres variantes: `ImpostorEngine`, `TimeEngine`, `DrawingEngine`.
PlayerService / AuthService	Vínculo socket ↔ jugador ↔ sala; identidad estable entre reconexiones; una segunda pestaña de la misma cuenta reemplaza a la primera.
RoomStore	Salas en memoria, listado de salas públicas y listas del workshop cargadas en cada sala. Limpia las salas abandonadas.
RoomsController / HealthController	`GET /rooms/public` (salas públicas) y `GET /health` (los clientes en una sala lo consultan para que el servidor no se duerma).
SocialGateway + SocialService	Solicitudes de amistad, presencia (cada 2 s avisa los cambios) e invitaciones a la sala.
WorkshopGateway + WorkshopService	Crear, editar, publicar, copiar, actualizar copias y dar like; valida listas (10 a 300 palabras) y presets.
ModerationGateway + ModerationService + AccountService	Reportes con evidencia del chat, acciones del panel de admin (suspender, ocultar, resolver), marcas de la cuenta (admin, suspensión, privacidad) y borrado de cuenta.

Componente	Responsabilidad
StatsService	Al terminar un juego, suma las estadísticas de los jugadores con cuenta (función `record_game_stats` de la base).

Tabla 13 — Componentes del Game Server

Componentes de la Aplicación Web

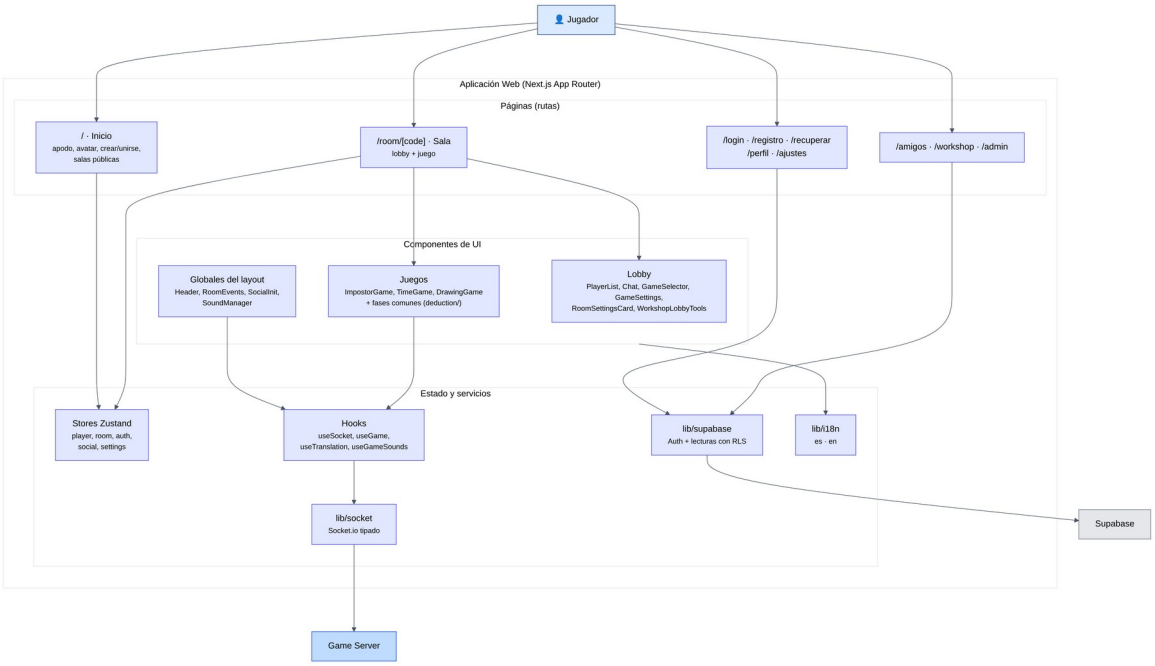


Figura 5 — C4 Nivel 3: componentes de la Aplicación Web

La aplicación web mantiene muy poco estado propio: el estado del juego llega del servidor y se guarda tal cual en los stores. `playerStore` guarda la identidad local (apodo, avatar, idioma); `roomStore`, la sala y el chat; `authStore`, la sesión de Supabase y el perfil; `socialStore`, amigos y solicitudes; `settingsStore`, sonido y avisos. Los eventos de la sala se escuchan en un componente global del layout (`RoomEvents`) para no perder mensajes mientras se navega hacia la sala. El socket es un *singleton* tipado con los mismos tipos que usa el servidor, de modo que un evento mal escrito no compila.

5.3 Nivel 3 — Código (C4 — Nivel 4): el contrato de los motores de juego

El punto de extensión más importante de la arquitectura es la interfaz `GameEngine`. Cada minijuego la implementa; el resto del sistema (salas, chat, transporte, estadísticas) sólo conoce la interfaz. Como los tres juegos son de deducción, lo común vive en la clase abstracta `DeductionEngine`, y cada juego sólo define la **actividad de la ronda**: dar pistas, medir un tiempo o dibujar.

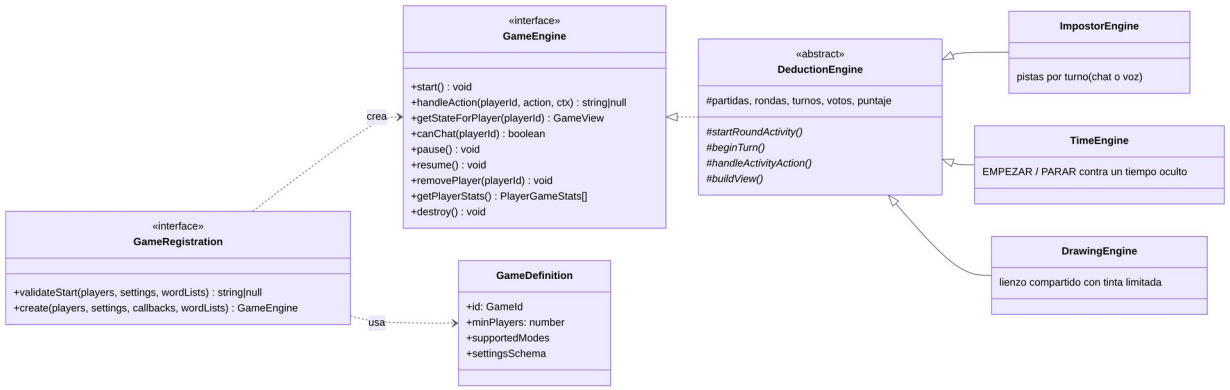


Figura 6 — C4 Nivel 4: interfaz GameEngine, motor base de deducción y juegos concretos

Con este contrato, agregar un minijuego nuevo requiere solamente:

1. Definir su `GameDefinition` (nombre, jugadores mínimos, modos y esquema de ajustes) y sus tipos de vista en `packages/shared`.
2. Implementar su motor: si es de deducción, extender `DeductionEngine` con los métodos de la actividad (`startRoundActivity`, `beginTurn`, `handleActivityAction`, `buildView`).
3. Registrarlo en `game.registry.ts`.
4. Crear sus componentes de interfaz en `apps/web/src/components/<juego>/` (las fases comunes ya están en `components/deduction/`).

El lobby lista automáticamente los juegos registrados y genera los controles de configuración a partir del `settingsSchema`; el servidor valida los ajustes contra el mismo esquema. Así se agregaron Impostor por tiempo e Impostor dibujo sin modificar salas, chat ni transporte (QS-7).

5.4 Estructura del repositorio

```
AmongLies/                        monorepo (npm workspaces + Turborepo)
├── apps/
│   ├── web/                      Aplicación Web (Next.js)
│   │   └── src/
│   │       ├── app/              /, /room/[code], /login, /registro, /recuperar, /perfil,
│   │       │                     /amigos, /workshop, /ajustes, /admin
│   │       ├── components/       lobby/, deduction/, impostor/, time/, drawing/, social/,
│   │       │                     workshop/, moderation/, sound/, layout/, ui/
│   │       ├── hooks/            useSocket, useGame, useTranslation, useGameSounds
│   │       ├── stores/           player, room, auth, social, settings
│   │       └── lib/              socket, supabase, i18n, sound, keepAlive, workshop, admin
│   └── server/                   Game Server (NestJS)
│       └── src/
│           ├── room/             gateway, store, validación, salas públicas
│           ├── chat/  player/  auth/  stats/
│           ├── game/             gateway, service, registry, settings, engines/
│           │                     (core, deduction, impostor, time, drawing)
│           ├── social/  workshop/  moderation/
│           └── health.controller.ts
├── packages/shared/              tipos, eventos, juegos, puntaje, listas de palabras,
│                                 filtro de insultos, tinta
├── supabase/migrations/          esquema de la base (con RLS)
├── e2e/                          pruebas de punta a punta (Playwright + bots)
├── docs/                         progreso, testing y esta documentación
└── turbo.json                    orden de build: shared → server, web
```

6. Vista de ejecución

Esta sección muestra cómo colaboran los bloques en los escenarios más importantes. Los diagramas de secuencia se leen de arriba hacia abajo; las flechas punteadas son respuestas.

6.1 Escenario 1 — Crear una sala y unirse

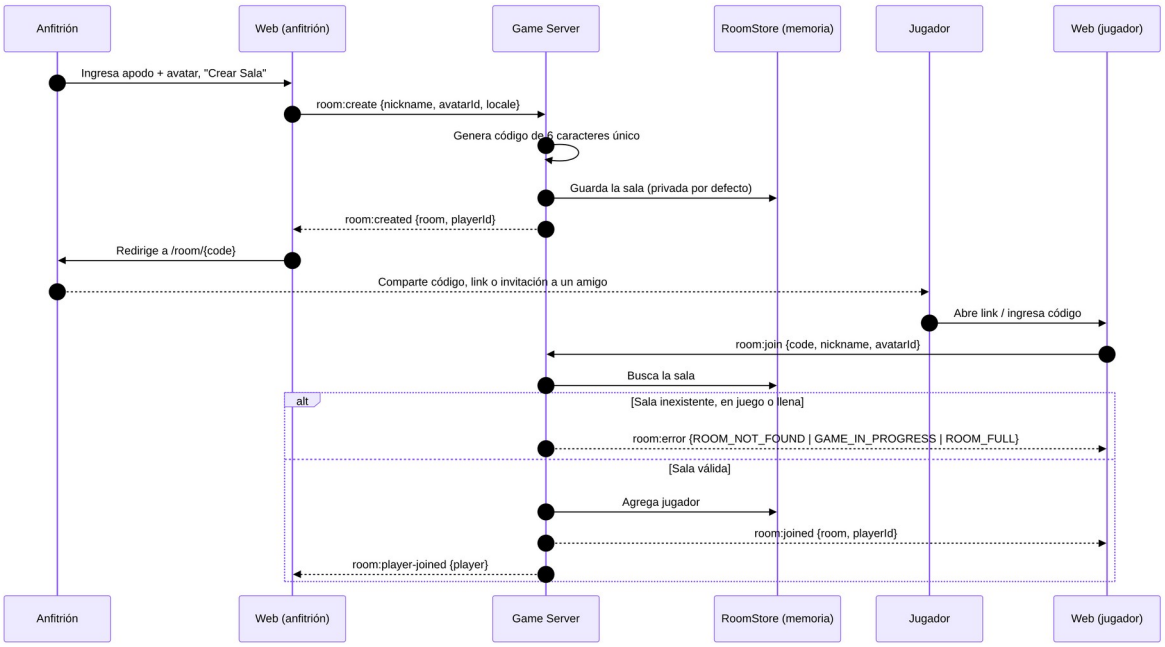


Figura 7 — Secuencia: creación de sala y unión de un jugador

Puntos de diseño: el servidor genera el código con un alfabeto sin caracteres ambiguos y verifica que no esté en uso; los errores de unión se devuelven con un **código** estable (`ROOM_NOT_FOUND`, `GAME_IN_PROGRESS`, `ROOM_FULL`, `NICKNAME_NOT_ALLOWED`) que el cliente traduce al idioma del usuario. Si el servidor está dormido (plan gratis), la web espera hasta 70 segundos mostrando "Despertando el servidor...".

6.2 Escenario 2 — Una ronda de El Impostor

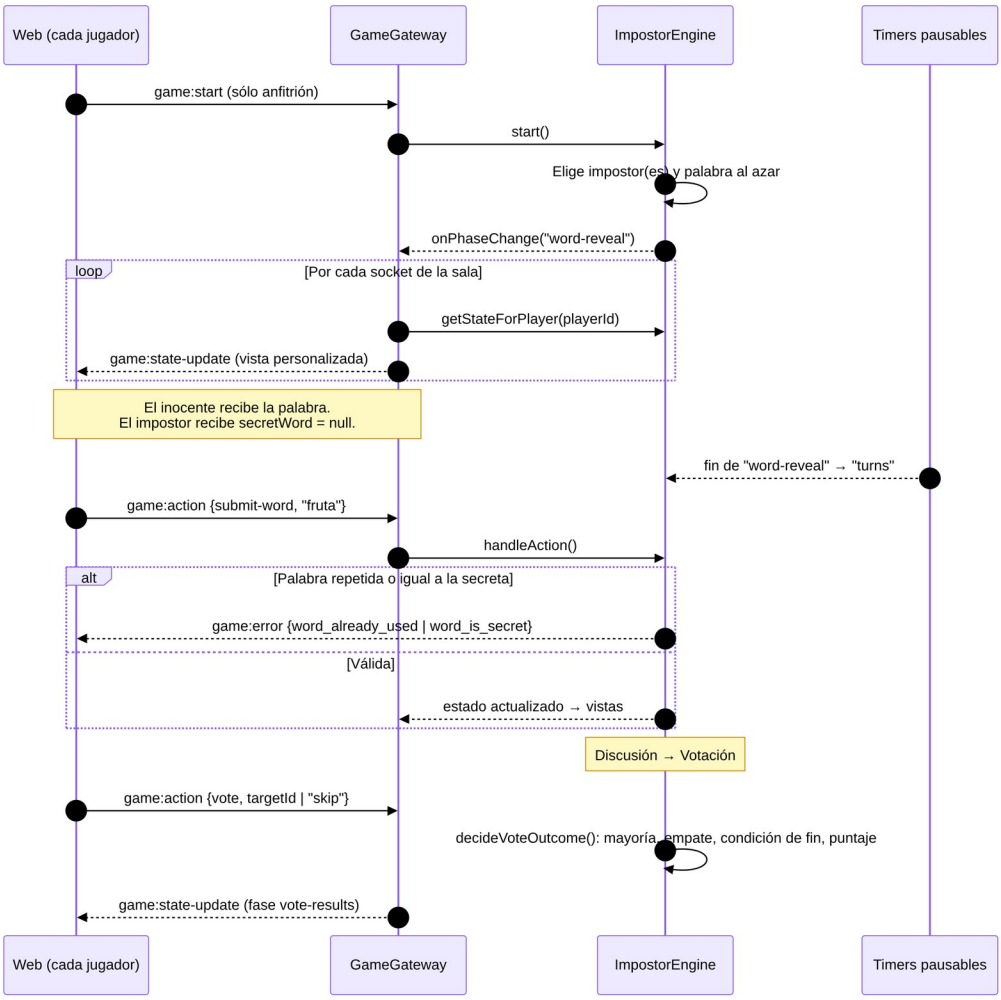


Figura 8 — Secuencia: ronda de El Impostor con servidor autoritativo

Lo central de este escenario es que, en cada cambio, el servidor **no difunde un único estado**: recorre los sockets de la sala y calcula `getStateForPlayer(playerId)` para cada uno. El impostor recibe `secretWord: null` (en el Tiempo, `targetMs: null`); los compañeros impostores sólo se revelan entre sí; los votos individuales se ocultan hasta la fase de resultados (y para siempre si el voto es secreto). En el Impostor dibujo los trazos viajan por un evento aparte (`game:draw`) para no reenviar todo el estado en cada movimiento del pincel.

6.3 Máquina de estados del motor El Impostor

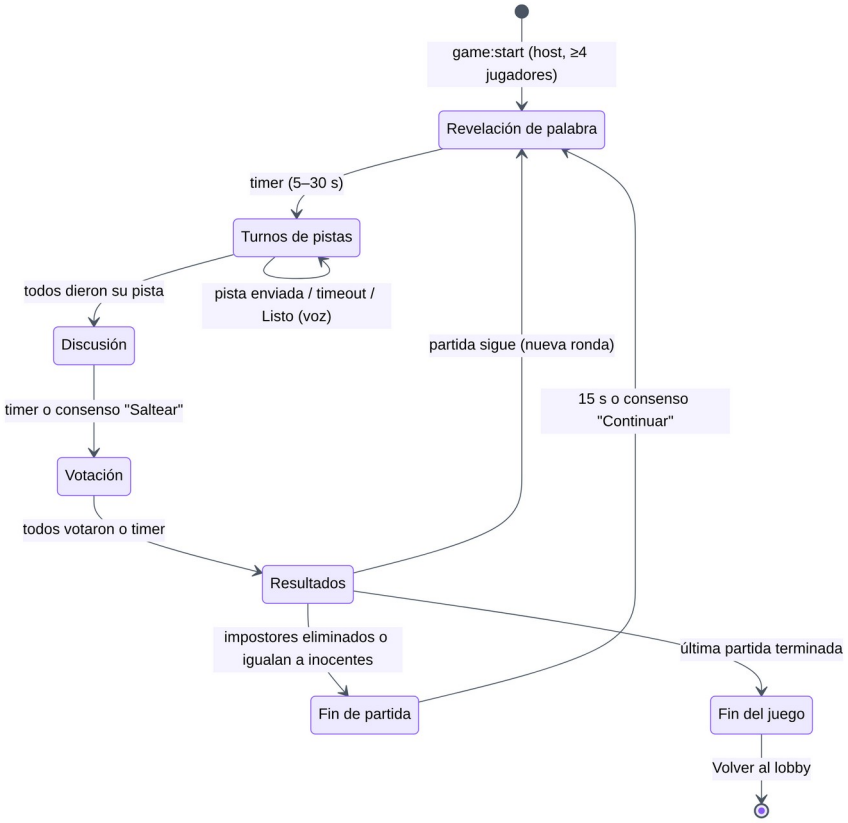


Figura 9 — Estados y transiciones de El Impostor

Estado	Sale por	Condición de salida
Revelación de palabra	Temporizador	`wordRevealTimeSeconds` (5–30 s). En el Tiempo, revelación de rol y del tiempo objetivo.
Turnos	Acción o temporizador	Cada jugador activo envía su pista (chat), toca "Listo" (voz), para el reloj (Tiempo) o termina de dibujar (Dibujo); si se agota el tiempo del turno, se registra.
Discusión	Temporizador o consenso	Termina el tiempo o **todos** los jugadores activos votan "Saltear discusión". Si el tiempo configurado es 0, se omite.
Votación	Todos votaron o temporizador	Los votos no emitidos cuentan como abstención. Se puede votar "saltear" si el anfitrión lo permite.
Resultados	Temporizador	Mayoría simple expulsa. Empate: nadie sale, re-votación entre los empatados o desempate al azar (configurable). Se suma el puntaje y se evalúa el fin de la partida (incluido el tope de rondas).
Fin de partida	15 s o consenso	Se revela el impostor y el puntaje; todos pueden votar "Continuar ya".
Fin del juego	Acción del usuario	Podio, resumen de todas las partidas y vuelta al lobby.

Tabla 14 — Transiciones del motor de deducción (común a los tres juegos)

Si un jugador se desconecta durante la partida, el motor se **pausa**: todos los temporizadores se congelan y se reanudan con el tiempo que les quedaba cuando vuelve (o cuando quien decide toca "Seguir sin esperar").

6.4 Escenario 3 — Fin del juego y registro de estadísticas

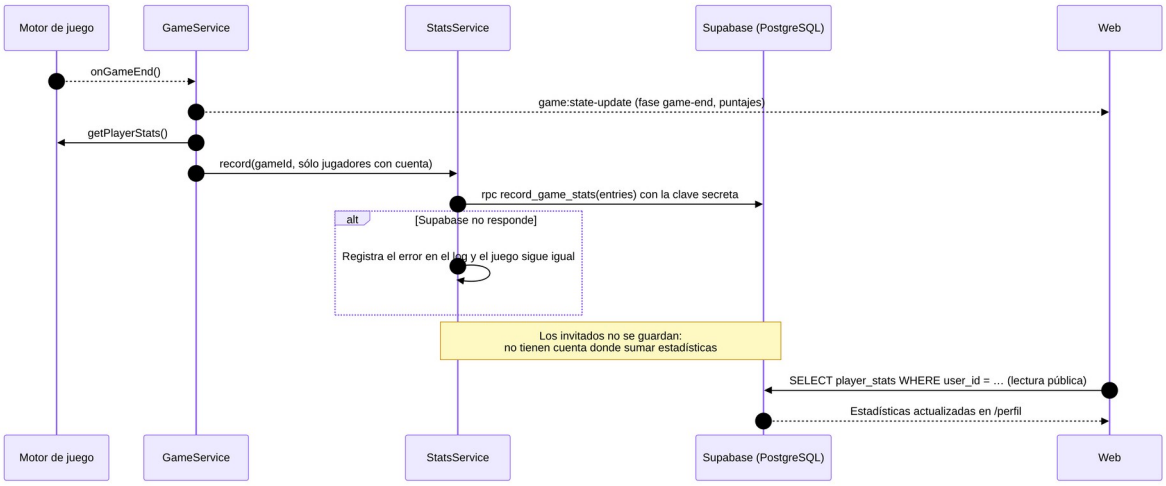


Figura 10 — Secuencia: registro de estadísticas al terminar el juego

La escritura de estadísticas es **asíncrona respecto del juego**: los jugadores ven la pantalla final aunque Supabase no responda. Si la escritura falla se registra el error en el log del servidor y el juego sigue; se aceptó perder esa suma antes que agregar una cola de reintentos.

6.5 Escenario 4 — Inicio de sesión y conexión autenticada

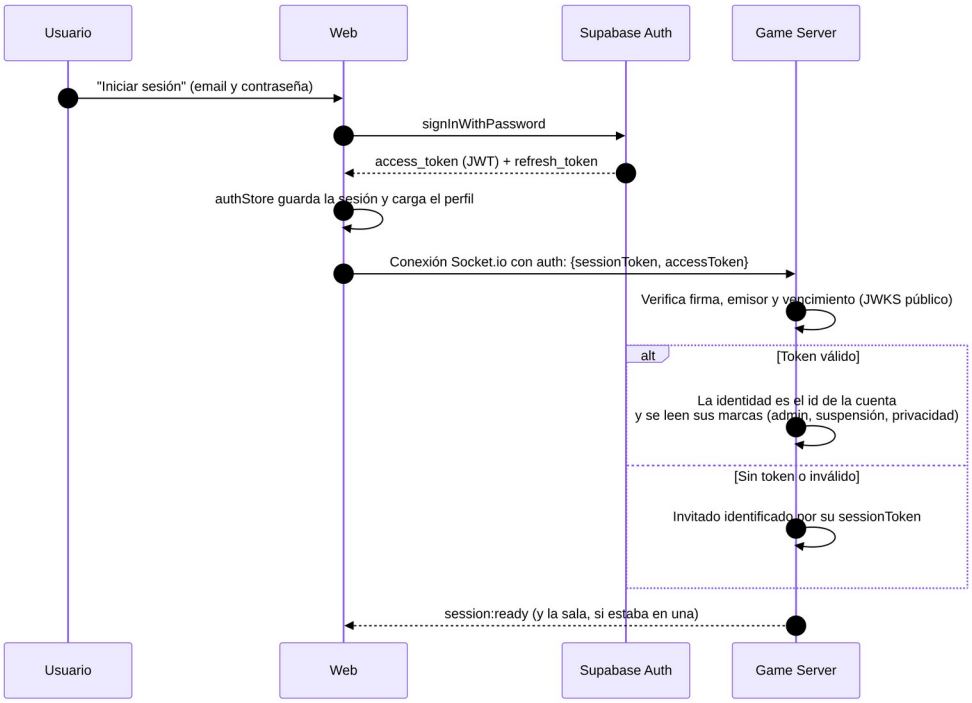


Figura 11 — Secuencia: login con Supabase y conexión autenticada

6.6 Escenario 5 — Desconexión y reconexión

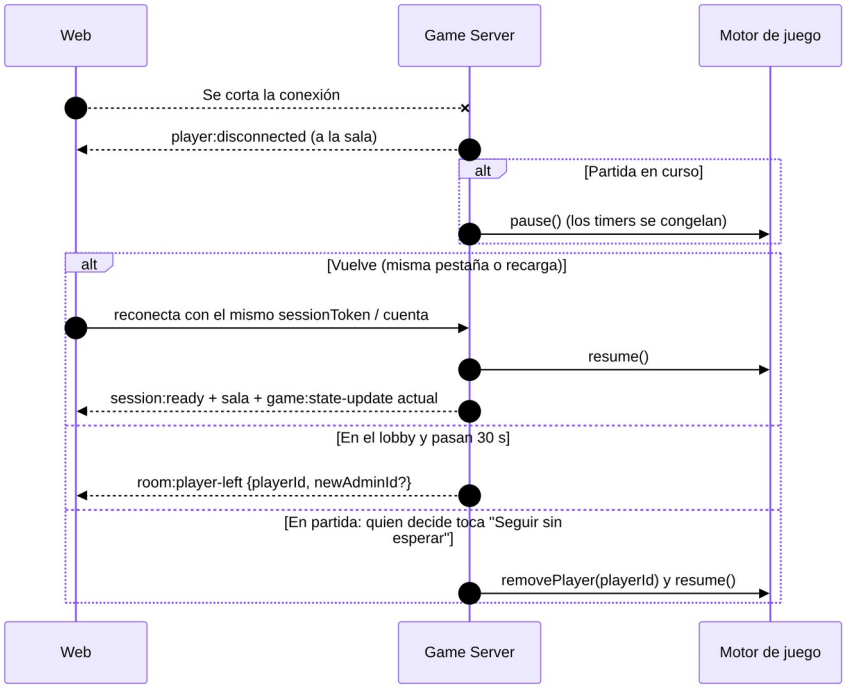


Figura 12 — Secuencia: desconexión, pausa y reconexión

6.7 Catálogo de eventos

Evento	Dirección	Quién puede emitirlo	Descripción
session:ready	S → C	Servidor	Identidad lista (invitado o cuenta); incluye la sala si estaba en una (reconexión).
room:create / room:join	C → S	Cualquiera no suspendido	Crea una sala (el emisor queda como anfitrión) o se une por código. Respuesta `room:created`, `room:joined` o `room:error`.
room:leave / room:kick / room:transfer-admin	C → S	Miembro / anfitrión	Salir, expulsar (el expulsado recibe `room:kicked`) o ceder el rol de anfitrión.
room:update-settings	C → S	Anfitrión	Visibilidad pública/privada, cupo e idioma de la sala.
room:updated / room:player-joined / room:player-left	S → C	Servidor	Cambios de la sala; `player-left` incluye `newAdminId` si se transfirió el rol.
player:disconnected / game:continue-without	S → C / C → S	Servidor / quien decide	Un jugador se desconectó (la partida se pausa); seguir sin esperarlo.
chat:send / chat:message	C ↔ S	Miembro no eliminado	Mensajes de chat (máx. 200 caracteres, insultos censurados).
game:select / game:update-settings / game:start	C → S	Anfitrión	Elegir minijuego, ajustar sus reglas e iniciar.
game:action	C → S	Jugador	`submit-word`, `ready`, `advance`, `start-clock`, `stop-clock`, `finish-drawing`, `skip-discussion`, `vote`, `skip-partida-end`…

Evento	Dirección	Quién puede emitirlo	Descripción
game:draw	C ↔ S	Quien dibuja	Trazos en vivo del Impostor dibujo; el servidor los recorta según la tinta y los reenvía a la sala.
game:state-update / game:error	S → C	Servidor	Vista personalizada del estado para ese jugador; errores con código.
social:*	C ↔ S	Cuentas	Buscar, pedir, aceptar, eliminar amigos; presencia; invitaciones a la sala.
workshop:*	C → S	Cuentas	Crear, editar, publicar, copiar, actualizar, dar like; usar una lista o un preset en el lobby.
report:create / admin: / *account:*	C → S	Todos / admins / cuentas	Reportar; acciones del panel de admin; refrescar o borrar la propia cuenta. `account:suspended` avisa una suspensión.

Tabla 15 — Contrato de eventos en tiempo real

7. Vista de despliegue

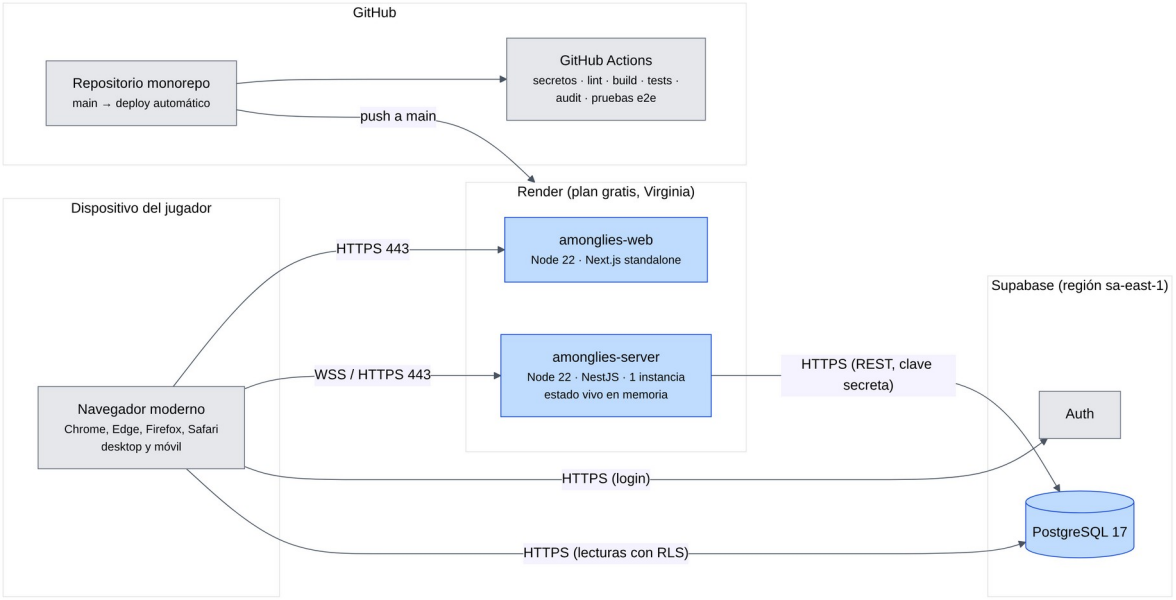


Figura 13 — Despliegue en producción

Nodo	Plataforma	Configuración	Justificación
amonglies-web	Render (Web Service, plan gratis, Virginia)	Node 22; `next build` con salida *standalone*; variables `NEXT_PUBLIC_SERVER_URL` y `NEXT_PUBLIC_SUPABASE_*`.	Deploy automático desde `main`; mismo proveedor que el servidor.
amonglies-server	Render (Web Service, plan gratis, Virginia), 1 instancia	Node 22; `node apps/server/dist/main`; variables `CLIENT_URL`, `SUPABASE_URL`, `SUPABASE_SECRET_KEY`; health check `/health`.	Admite procesos de larga duración y WebSockets (RT-2).
PostgreSQL + Auth	Supabase, región sa-east-1 (São Paulo)	RLS en todas las tablas; la clave secreta sólo en el servidor; 6 migraciones versionadas en el repo.	Auth y base en un mismo servicio con plan gratuito.
CI	GitHub Actions	En cada push y pull request: secretos, lint, build, tests con cobertura, `npm audit` y pruebas de punta a punta.	Evita integrar a `main` código que rompe el contrato o las partidas.

Tabla 16 — Nodos de despliegue

La configuración de los dos servicios de Render está versionada en `render.yaml` (en la raíz del repositorio); las claves secretas se cargan a mano en el panel de Render.

> **El plan gratis de Render duerme el servidor**
>
> A los 15 minutos sin tráfico HTTP el servidor se apaga y tarda alrededor de un minuto en despertar; al despertar, las salas en memoria se pierden. La aplicación lo mitiga: los clientes que están en una sala consultan `GET /health` cada 4 minutos (el servidor no se duerme mientras alguien juega); si igual se pierde la sala, se avisa "La sala ya no existe" y se vuelve al inicio; y al crear o unirse se espera al servidor con el aviso "Despertando el servidor...". El plan pago de Render no duerme el servidor.

Entornos

Entorno	Cómo se levanta	Datos
Local	`npm install` y `npm run dev:all` (Turborepo compila shared antes de las apps). Sin la clave de Supabase el servidor funciona igual para invitados.	Proyecto Supabase del equipo (o ninguno).
CI	Build de producción; servidor y web levantados en el runner; pruebas de punta a punta de invitados.	Sin base de datos.
Producción	Push a `main` → Render construye y despliega web y server automáticamente.	Datos reales.

Tabla 17 — Entornos

8. Conceptos transversales

8.1 Modelo de dominio

Varios términos del juego tienen un significado preciso en el sistema (ver también el glosario, §12):

- Una **sala** agrupa jugadores y tiene un **anfitrión**. En la sala se juega un **juego** de los tres disponibles.
- Un juego tiene N **partidas** (configurable 1–10). En cada partida los impostores son los mismos; se eligen de nuevo al empezar la siguiente.
- Una partida tiene varias **rondas**: actividad (pistas, tiempos o dibujo), discusión y votación. Termina cuando todos los impostores fueron expulsados o cuando quedan tantos impostores como inocentes (o al llegar al tope de rondas, si hay uno: ganan los impostores).
- En El Impostor cada ronda tiene una palabra nueva; en el Impostor dibujo hay **una palabra y un lienzo por partida**, y cada ronda se dibuja sobre el mismo dibujo; en el Impostor por tiempo cada ronda tiene un **tiempo objetivo** nuevo.
- El **puntaje** se calcula por ronda y por partida, y se muestra al terminar cada partida (antes delataría al impostor).

8.2 Diseño de datos

El sistema separa deliberadamente dos clases de datos con necesidades opuestas:

	Estado vivo	Datos persistentes
Ejemplos	Salas, jugadores conectados, fase actual, votos, temporizadores, chat, lienzo.	Cuentas, perfiles, estadísticas, amistades, workshop, reportes.
Vida útil	Minutos u horas; desaparece al cerrar la sala.	Indefinida.
Patrón de acceso	Muchas escrituras por segundo por sala.	Pocas escrituras; lecturas con filtros y agregados.
Requisito crítico	Latencia mínima.	Integridad y control de acceso.
Almacén	**Memoria** del proceso servidor (una instancia).	**PostgreSQL** en Supabase.

Tabla 18 — Por qué hay dos almacenes de datos

8.2.1 Estado vivo en memoria

`RoomStore` guarda un `Map<código, sala>` con los jugadores, el anfitrión, los ajustes, el chat (últimos mensajes) y las listas del workshop elegidas; `GameService` guarda el motor activo de cada sala; `PlayerService` vincula cada socket con su identidad estable (el `sessionToken` del invitado o el id de la cuenta) para poder reconectar. Las salas sin jugadores se limpian solas. No se lleva a Redis (ADR-05): con una instancia alcanza para el uso real del producto y Redis agregaría costo y complejidad.

8.2.2 Modelo persistente en PostgreSQL

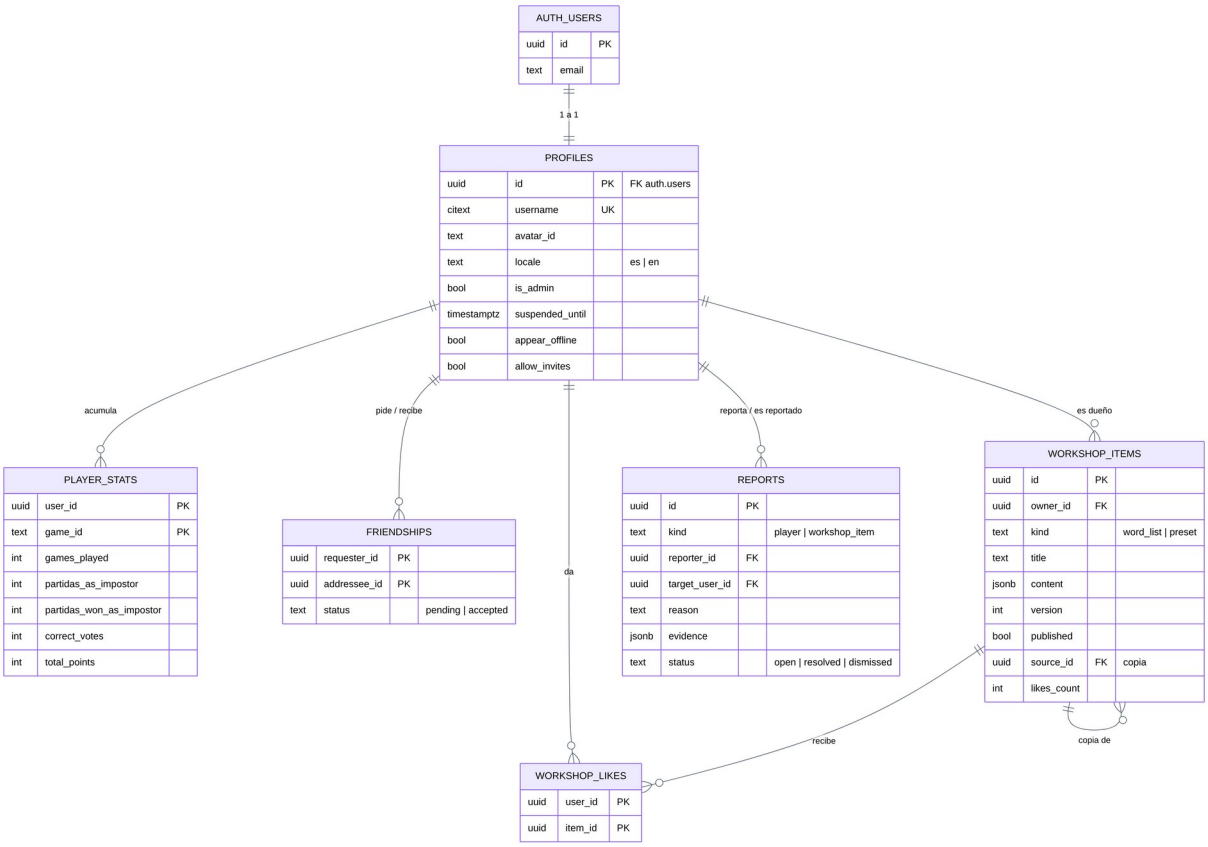


Figura 14 — Modelo entidad–relación de los datos persistentes

Tabla	Propósito	Decisiones de diseño
profiles	Datos del usuario.	Relación 1 a 1 con `auth.users` (un trigger crea el perfil al registrarse). `username` es citext único: evita "Juan" y "juan" duplicados. Marcas de admin, suspensión y privacidad.
player_stats	Estadísticas acumuladas por usuario y juego.	Una fila por (usuario, juego). Las suma la función `record_game_stats`, que sólo puede llamar el servidor.
friendships	Solicitudes y amistades.	Una fila por par, sin importar quién pidió (índice único). Estado `pending` / `accepted`.
workshop_items	Listas de palabras y presets.	`content` jsonb (palabras o ajustes). `version` y `source_id`/`source_version` para que una copia sepa si el original cambió.
workshop_likes	Likes del workshop.	Una fila por (usuario, ítem); el contador `likes_count` lo mantiene un trigger.
reports	Reportes de jugadores y contenido.	Guarda motivo, detalle y evidencia (chat); sólo la lee y escribe el servidor.

Tabla 19 — Tablas persistentes

8.2.3 Políticas de acceso (Row Level Security)

- **RLS activado en todas las tablas** y revisado con los *advisors* de seguridad de Supabase después de cada migración.

- `profiles`: lectura pública; cada usuario sólo puede modificar **su** avatar, idioma y privacidad (permiso por columna: no puede hacerse admin ni levantar su suspensión).
- `player_stats` y `workshop_items` publicados: lectura pública. `friendships`: sólo las dos personas involucradas. Lo privado del workshop: sólo su dueño. `reports`: nadie desde el cliente.
- Las escrituras las hace únicamente el Game Server con la clave secreta, que nunca llega al navegador (ADR-15). Una prueba de punta a punta verifica con el token de un usuario común que no puede saltearse estas reglas.

8.3 Contrato de comunicación

- Todos los eventos y sus datos están tipados en `packages/shared/src/types/events.ts` (`ClientEvents`, `ServerEvents`). Cliente y servidor importan el mismo archivo: si alguien cambia un evento, el otro lado deja de compilar (QS-10).
- Los errores viajan como **códigos** (`ROOM_FULL`, `word_already_used`) y no como texto; el cliente los traduce. Así el servidor no depende del idioma del usuario.
- Las acciones de juego usan un único evento `game:action` con `{ type, payload }`, para que un minijuego nuevo no requiera eventos nuevos en el transporte.

8.4 Seguridad e integridad del juego

Amenaza	Control
Un impostor inspecciona el tráfico o el estado del navegador para ver la palabra (o el tiempo objetivo).	El dato **nunca se envía** al impostor: la vista se calcula por jugador en el servidor.
Un jugador envía acciones fuera de turno, vota dos veces o dibuja de más.	El motor valida fase, turno, rol y si ya votó; el servidor recorta los trazos a la tinta disponible.
Un jugador no anfitrión intenta expulsar, configurar o iniciar.	Los gateways comparan el jugador de la sesión con el anfitrión de la sala.
Datos malformados o excesivos.	Validación de cada evento (tipos, rangos, claves extra) contra los esquemas; mensajes recortados a 200 caracteres; apodos de 2–16.
Insultos y abuso.	Filtro de insultos compartido (chat y pistas censurados; apodos, usuarios y workshop rechazados), reportes con evidencia, suspensiones; máximo 5 reportes cada 10 minutos.
Suplantación de usuario registrado.	JWT firmado por Supabase verificado con su JWKS público; sin token válido, la sesión es de invitado.
Escalada de privilegios desde el cliente (hacerse admin).	Permisos por columna en `profiles`; el servidor verifica en la base que el usuario sea admin en cada acción del panel.
Lectura de datos de otros usuarios.	Row Level Security en PostgreSQL.
Intercepción de tráfico.	HTTPS/WSS en todos los canales; CORS restringido al dominio del frontend (`CLIENT_URL`).
Filtración de secretos.	Claves sólo en variables de entorno del servidor; un escaneo de secretos en el hook pre-commit y en el CI.
Dependencias vulnerables.	`npm audit` de producción en el CI: el build falla con vulnerabilidades altas o críticas.

Tabla 20 — Amenazas y controles

8.5 Manejo del tiempo

Todos los temporizadores (ver palabra, turno, discusión, votación, resultados, fin de partida) viven en el **servidor**, como *timers pausables*: si alguien se desconecta se congelan y al reanudar siguen con el tiempo que les quedaba. El cliente sólo muestra una cuenta regresiva. En el Impostor por tiempo, el reloj de cada jugador se mide en el navegador (`performance.now()`) para no sumar la latencia de la red, pero el servidor **no acepta más tiempo del que realmente pasó** desde que el jugador tocó EMPEZAR.

8.6 Identidad y sesiones

- **Invitado**: el navegador genera un `sessionToken` por pestaña (en `sessionStorage`), que nunca se difunde a otros jugadores; el servidor genera el `playerId` público. Recargar la página conserva la identidad.
- **Registrado**: la identidad es el id de la cuenta, tomado del JWT. El apodo por defecto es su nombre de usuario y sus resultados se suman a su perfil. Una segunda pestaña con la misma cuenta reemplaza a la primera.
- **Reconexión**: al volver, `session:ready` restaura la sala y el estado del juego. En el lobby, si no vuelve en 30 segundos, sale de la sala; en la partida se espera (pausada) hasta que vuelva o hasta que quien decide toque "Seguir sin esperar".

8.7 Internacionalización

La interfaz usa diccionarios por idioma (**español rioplatense e inglés**); el idioma se detecta del navegador la primera vez y luego se recuerda (y, con cuenta, se guarda en el perfil). El idioma de la **sala** (elegido por el anfitrión) define las listas de palabras; el idioma de la **interfaz** es personal de cada jugador. Hay 8 listas de palabras por idioma (40–50 palabras cada una) y las listas del workshop tienen su propio idioma. Se priorizó traducir completos estos dos idiomas antes que ofrecer más idiomas a medias.

8.8 Manejo de errores y retroalimentación

- Validaciones inmediatas en el cliente (apodo, formato del código, cantidad de palabras de una lista) para responder sin ir al servidor.
- Errores del servidor con código traducible y mensaje visible en pantalla.
- Al crear o unirse, si el servidor no responde se muestra "Despertando el servidor…" y se espera hasta 70 segundos (plan gratis de Render).
- Si el servidor se reinició y la sala ya no existe, se avisa "La sala ya no existe" y se vuelve al inicio, en lugar de dejar al jugador en una sala fantasma.
- Indicador de "Desconectado del servidor" y pantalla de pausa durante la partida cuando falta alguien.

8.9 Escalabilidad

Decisión: una sola instancia. Todo el estado vivo está en memoria de un proceso. Es adecuado para decenas de salas simultáneas —muy por encima del uso real del producto— y mantiene la operación simple y gratuita. No permite agregar instancias: dos réplicas no compartirían salas.

Si en el futuro hiciera falta escalar, el camino está identificado y no requiere reescribir los motores: llevar `RoomStore` a Redis (detrás de la misma interfaz), usar el adaptador Redis de Socket.io para difundir entre réplicas y ejecutar el motor de cada sala en una sola réplica. Se postergó (ADR-05) porque su costo no se justifica con el tráfico actual.

8.10 Observabilidad

- Logs del Logger de NestJS (errores de Supabase, estadísticas que no se guardaron, inicio del servidor), visibles en el panel de Render.
- Endpoint `GET /health` para el health check de Render y para mantener despierto el servidor.
- Los *advisors* de seguridad y rendimiento de Supabase se revisan después de cada migración.

8.11 Estrategia de pruebas

Nivel	Qué se prueba	Herramienta y resultado
Unitarias	Reglas de los tres juegos sin red y con timers simulados: votos y empates (con TDD), fin de partida, puntaje, tinta, tiempo objetivo, validación de ajustes, salas, amigos, workshop, moderación y filtro de insultos.	Jest: **291 tests**; cobertura de ramas mínima exigida en la lógica crítica (100 % en votos y reglas). Prueba de mutación manual: 8 de 8 mutantes detectados.
De punta a punta (invitados)	Partidas reales de los tres juegos en Chromium con bots por socket, en escritorio y celular; recarga, reconexión, reglas que valida el servidor, salas públicas, chat, servidor que se reinicia.	Playwright + socket.io-client: 12 suites (≈120 checks). **Corren en el CI** contra el build recién compilado.
De punta a punta (cuentas)	Login, estadísticas, amigos, workshop, moderación (incluido RLS con el token de un usuario común) y ajustes, con cuentas temporales que se borran al terminar.	5 suites (81 checks), contra Supabase.
Manual	Partidas completas con 4 navegadores contra producción en cada fase; las capturas del Manual de usuario salen de esas partidas.	—

Tabla 21 — Estrategia de pruebas

Además, un **hook pre-commit** busca secretos, corre el linter y los tests antes de cada commit, y el **CI** repite todo en cada push. El detalle (ciclo TDD, casos borde, mutación y los 29 bugs que se encontraron en el propio código) está en `docs/TESTING.md` del repositorio.

8.12 Diseño de la interfaz: decisiones y justificación

La interfaz se diseñó primero en Figma (pantallas del Impostor en escritorio y celular) y después directamente en código con los mismos *tokens* de diseño, para no duplicar trabajo en las pantallas nuevas (amigos, workshop, ajustes, admin, Tiempo y Dibujo). Cada decisión de diseño responde a un objetivo de calidad (§1.4): **baja fricción** (O1), **juego justo** (O3) y **accesible desde el celular** (O6).

Identidad visual "Neon Noir"

Token	Valor	Uso	Por qué
Fondo	#0A0A1A	Fondo de todas las páginas.	Oscuro: el juego se usa de noche y en reuniones, cansa menos la vista y hace resaltar los colores con significado. Refuerza el tono de misterio.
Superficies	#161629 / #1E1E3A	Tarjetas, paneles, campos.	Escalones de luminosidad para separar capas sin bordes gruesos.
Primario (violeta)	#8B5CF6	Acciones principales: Crear Sala, Iniciar Juego, Votar en el lobby, opciones elegidas.	Un solo color para "lo que hay que tocar" guía la vista.

Token	Valor	Uso	Por qué
Acento (rosa)	#EC4899	Todo lo del impostor: "¡Sos el impostor!", etiqueta IMPOSTOR, "Lies" del logo.	El jugador reconoce su rol de un vistazo, sin leer.
Éxito (verde)	#10B981	Inocentes, palabra secreta, confirmaciones, "Los jugadores ganan".	Asociación universal con "correcto".
Advertencia (ámbar)	#F59E0B	Temporizadores, discusión, acceso Admin, avisos.	Urgencia sin alarma.
Peligro (rojo)	#EF4444	Votación, expulsión, errores, borrar cuenta.	Acciones irreversibles o con consecuencias.
Texto	#F8FAFC / #94A3B8 / #475569	Principal, secundario, apagado.	Contraste alto para el texto principal sobre el fondo oscuro.

Tabla 22 — Paleta de colores y su significado

Tipografías: *Space Grotesk* para títulos y el logo (personalidad, de juego), *Inter* para el texto (muy legible en tamaños chicos) y *JetBrains Mono* para el código de sala, los números de los ajustes y los tiempos (cifras de ancho fijo: un código como "8UQ8FS" no se confunde y los tiempos no "bailan" al cambiar). Bordes redondeados grandes (tarjetas de 16–24 px) para un estilo amable, de juego casual.

Decisiones de diseño de la interacción

ID	Decisión	Alternativa descartada	Justificación
DD-01	Entrar con apodo y avatar, sin registro; el link de la sala lleva directo a "Unirse".	Registro obligatorio.	O1: cada paso extra pierde jugadores. La cuenta queda como mejora opcional.
DD-02	Mobile-first: una columna en el celular, header en menú ☰ (< 768 px), chat como botón flotante con globo de no leídos (< 1024 px).	Versión aparte para celular.	O6: la mayoría juega desde el celular en una reunión. Una sola base de código, probada a 360 px sin scroll horizontal.
DD-03	El rol se comunica con color y texto a la vez (rosa + "¡Sos el impostor!"; verde + palabra).	Sólo texto.	Se entiende en un segundo; no depende sólo del color (daltonismo).
DD-04	Revelación escalonada de los resultados (quién sale → ¿era? → detalle).	Mostrar todo junto.	Genera suspenso, el momento más divertido del juego.
DD-05	El puntaje se muestra al terminar cada partida, no por ronda; la palabra sólo se revela si sale un impostor.	Puntaje en vivo.	O3: los puntos por ronda delatarían al impostor ("sobrevivió +2"); revelar la palabra al expulsar a un inocente se la regalaría al impostor.
DD-06	Consenso para saltear (discusión, fin de partida): hace falta que **todos** voten.	Que el anfitrión decida.	Nadie queda afuera de la discusión por apuro de otro.
DD-07	Controles del anfitrión visibles para todos pero bloqueados para el resto.	Ocultarlos.	Todos ven las reglas que se van a jugar (transparencia) y saben quién decide.
DD-08	Dentro de la sala se ocultan cuenta, amigos y workshop del encabezado.	Encabezado completo.	Evita salir de una partida o cambiar de cuenta sin querer.
DD-09	Acciones destructivas con confirmación proporcional: borrar un ítem pide confirmación; borrar la cuenta exige escribir el usuario.	Confirmar siempre igual.	Cuanto más grave e irreversible, más fricción. Expulsar no pide confirmación porque se puede volver a entrar.

ID	Decisión	Alternativa descartada	Justificación
DD-10	Validación inmediata en el cliente y mensajes de error del servidor traducidos, junto al campo.	Errores genéricos.	El usuario sabe qué corregir sin esperar ni adivinar.
DD-11	Estados de espera explícitos: "Esperando a X…", "N/M votaron", "Despertando el servidor…", pausa con nombres.	Pantallas sin indicación.	En un juego en red el silencio parece un error.
DD-12	Sonido sintetizado y opcional (efectos y música por separado, con volumen).	Sin sonido / archivos de audio.	Avisa "te toca" aunque no se mire la pantalla; se puede apagar (ADR-19).
DD-13	Español rioplatense con voseo ("Creá", "Tocá") e inglés completo.	Español neutro.	Es el idioma del público principal; un tono cercano encaja con un juego entre amigos.
DD-14	Salas privadas por defecto; sala pública como opción explícita.	Públicas por defecto.	Privacidad y menos abuso (R-4).
DD-15	Dibujo con tinta limitada y sin deshacer.	Dibujo libre.	Obliga a pensar cada trazo y deja lugar al impostor; deshacer permitiría "borrar pistas".
DD-16	Reloj oculto en el Impostor por tiempo (no se ve ni el propio).	Reloj visible.	Si se viera, el impostor copiaría el tiempo de los demás sin deducir.

Tabla 23 — Decisiones de diseño de la interacción

Accesibilidad

- Todos los campos tienen su etiqueta asociada (se puede tocar la etiqueta y los lectores de pantalla la leen); los botones de sólo ícono tienen nombre accesible ("Echar a Sofi", "Abrir chat", "Reportar a Mica").
- Roles ARIA donde corresponde: interruptores (`switch`), pestañas (`tablist`), selector de idioma (`listbox`), avisos (`status`). El idioma de la página sigue al elegido.
- Las tarjetas de votación se pueden elegir con el teclado (Enter); el chat se envía con Enter.
- Áreas táctiles amplias en el celular y `touch-action: none` en el lienzo para dibujar sin que la página se desplace.
- El significado nunca depende sólo del color: el rol, los errores y los resultados también se escriben.

Animación y retroalimentación

Framer Motion se usa con un propósito: transiciones entre fases (para que se note que el juego avanzó), la revelación escalonada (DD-04) y los avisos que entran y salen. Las animaciones son cortas (≤ 0,6 s) para no demorar el juego.

9. Decisiones de arquitectura

Cada decisión se documenta como un **ADR** (*Architecture Decision Record*): el contexto que la motivó, las alternativas que se evaluaron, los criterios usados, la decisión y sus consecuencias —incluidas las negativas—. El foco no está en la tecnología elegida sino en **por qué** se eligió frente a las otras.

9.1 Criterios de evaluación

Para que las decisiones sean comparables, el equipo definió criterios derivados de los objetivos de calidad (§1.4) y de las restricciones (§2), con un peso relativo:

Criterio	Peso	Origen
K1 Latencia / tiempo real	5	Objetivo de calidad 1, O2
K2 Integridad del juego	5	Objetivo de calidad 2, O3
K3 Fricción para el usuario	4	Objetivo de calidad 3, O1
K4 Extensibilidad	3	Objetivo de calidad 4, O4
K5 Costo de operación	3	RT-4, O7
K6 Conocimiento del equipo / curva de aprendizaje	3	RO-1, RO-2
K7 Simplicidad operativa	2	RO-1

Tabla 24 — Criterios de decisión y pesos

En las decisiones con más alternativas se incluye una matriz de puntaje (1 = malo, 5 = excelente), ponderada con estos pesos.

ADR-01 — Comunicación en tiempo real con Socket.io

Contexto. Todos los jugadores de una sala deben enterarse de cada cambio (turno, voto, mensaje) en menos de 150 ms. El servidor necesita *empujar* información sin que el cliente la pida.

Alternativa	K1 (×5)	K2 (×5)	K5 (×3)	K6 (×3)	K7 (×2)	Total
REST + polling cada 1 s	1	4	2	5	5	**60**
Server-Sent Events + REST	4	4	4	3	3	**75**
WebSocket nativo (`ws`)	5	4	5	3	2	**78**
Socket.io	5	4	5	4	4	**85**
Servicio gestionado (Firebase RTDB / Supabase Realtime)	4	2	3	3	5	**63**

Tabla 25 — ADR-01: matriz de decisión de la comunicación

Decisión. Socket.io. Frente a WebSocket nativo aporta, sin código propio, *rooms* para difundir por sala, reconexión automática, respaldo por long-polling en redes que bloquean WebSockets y un adaptador Redis para escalar. SSE se descartó por ser unidireccional (las acciones seguirían necesitando HTTP) y polling por su latencia y costo. Los servicios gestionados se descartaron porque trasladan la lógica al cliente, lo que choca con K2.

Consecuencias. (+) Tiempo real bidireccional con una única conexión; integración nativa con NestJS. (−) Protocolo propio: el cliente debe usar `socket.io-client`; si se usa long-polling con varias réplicas hace falta afinidad de sesión (se mitiga priorizando el transporte WebSocket).

ADR-02 — Servidor autoritativo

Contexto. El juego depende de información oculta y de reglas (turnos, votos) que alguien debe hacer cumplir.

Alternativas. (a) Cliente autoritativo: cada navegador calcula el estado y lo comparte; (b) anfitrión autoritativo: el navegador del anfitrión hace de servidor (P2P/WebRTC); (c) **servidor autoritativo**.

Decisión. (c). Es la única opción en la que un jugador no puede leer ni alterar lo que no le corresponde (K2), y evita que el juego dependa de la conexión del anfitrión.

Consecuencias. (+) Integridad garantizada por diseño; los clientes son simples. (−) Toda la carga está en el servidor y cada acción hace un viaje de ida y vuelta, lo que refuerza la importancia de ADR-01 y ADR-05.

ADR-03 — Monolito modular con NestJS

Contexto. El servidor tiene varios dominios (salas, chat, juego, estadísticas) y debe crecer con nuevos minijuegos, mantenido por un equipo chico.

Alternativa	Ventaja principal	Motivo de descarte
Express “plano” + Socket.io	Mínimo, conocido.	Sin estructura: con tres juegos el código de eventos se vuelve difícil de mantener (K4).
Colyseus (framework de game servers)	Resuelve salas y sincronización de estado.	Modelo de estado propio y más curva de aprendizaje (K6); acopla todo el sistema a una librería de nicho.
Microservicios (salas, juego, chat)	Escalado y despliegue independientes.	Costo operativo y de red desproporcionado para el tamaño del equipo (K5, K7); el juego necesita salas y chat con latencia mínima.
NestJS modular	Módulos por dominio, inyección de dependencias, gateways WebSocket nativos, TypeScript.	— (elegida)

Tabla 26 — ADR-03: alternativas para el servidor

Consecuencias. (+) Límites claros entre módulos; servicios fáciles de probar por inyección de dependencias; si en el futuro un módulo necesitara separarse (por ejemplo estadísticas) ya está aislado. (−) Más código ceremonial (decoradores, módulos) que Express.

ADR-04 — Next.js (App Router) para la aplicación web

Contexto. La web tiene páginas muy interactivas (la sala) y páginas de contenido (inicio, salas públicas, perfiles).

Alternativas. SPA con Vite + React; Next.js.

Decisión. Next.js. Las pantallas de juego se renderizan en el cliente (`"use client"`), pero la página de inicio, el explorador de salas públicas y los perfiles se benefician del renderizado en servidor (primer pintado más rápido, K3) y del enrutado por archivos. La salida *standalone* produce un servidor Node autocontenido fácil de desplegar.

Consecuencias. (+) Rendimiento de carga inicial; convenciones claras. (−) Build más pesado que Vite; hay que marcar explícitamente los componentes de cliente.

ADR-05 — Estado vivo en memoria (sin Redis)

Contexto. El estado de una partida cambia varias veces por segundo y sólo importa mientras la sala existe. Se evaluó guardarlo en Redis para tener varias réplicas y sobrevivir a reinicios.

Alternativa	K1 (×5)	K5 (×3)	K7 (×2)	Escala horizontal	Sobrevive reinicio
Memoria del proceso	5	5	5	No	No
Redis	5	3	3	Sí	Sí
PostgreSQL	2	4	4	Sí	Sí

Tabla 27 — ADR-05: dónde guardar el estado vivo

Decisión. Memoria, también en la versión final. Redis se descartó al construirla: no hay un plan gratuito de Redis que convenga junto a Render, exigía resolver qué réplica ejecuta cada partida y el tráfico real entra holgado en una instancia. PostgreSQL se descarta para este uso: una escritura transaccional por cada voto o trazo agrega latencia sin beneficio, porque el dato es efímero.

Consecuencias. (+) Latencia mínima, cero infraestructura extra, operación simple. (−) Un reinicio del servidor pierde las salas activas (R-1): se mitiga manteniendo despierto el servidor mientras hay jugadores y avisando con claridad si pasa. `RoomStore` tiene una interfaz propia, por lo que cambiar de almacén no obliga a tocar los gateways.

ADR-06 — PostgreSQL (Supabase) para datos persistentes

Contexto. La versión final guarda usuarios, estadísticas, amistades, el workshop y reportes. Los datos son relacionales (un usuario tiene muchos amigos, ítems y likes; un ítem tiene copias) y las estadísticas son agregaciones por usuario y juego.

Alternativas. MongoDB (documentos); Firebase Firestore; **PostgreSQL gestionado en Supabase**.

Decisión. PostgreSQL en Supabase. Las relaciones N:M y los agregados se expresan naturalmente en SQL, con integridad referencial. Supabase suma, en un mismo servicio gratuito, autenticación y Row Level Security (K5, K7). Además es el modelo que el equipo conoce de la carrera (K6). Se descartó MongoDB porque las estadísticas requieren cruzar colecciones, y Firestore por su modelo de costos por lectura y por acoplar el cliente a la base.

Consecuencias. (+) Consultas de estadísticas simples y consistentes; seguridad declarativa con RLS. (−) Dependencia de un proveedor; el plan gratuito pausa proyectos inactivos (riesgo R-6).

ADR-07 — Cuentas opcionales (invitado primero) con Supabase Auth

Contexto. El objetivo O1 pide jugar sin registrarse, pero O5 pide guardar progreso.

Alternativas. (a) Registro obligatorio; (b) autenticación propia (bcrypt + JWT); (c) **cuenta opcional con Supabase Auth**; (d) Auth0/Clerk.

Decisión. (c). El registro obligatorio contradice el objetivo con mayor impacto en la adopción (K3). Implementar autenticación propia implica manejar contraseñas, recuperación y verificación de email, con mayor riesgo de seguridad. Auth0/Clerk resuelven lo mismo que Supabase pero serían un proveedor más. El servidor sólo **verifica** el JWT; si no hay token, el usuario juega como invitado.

Consecuencias. (+) Entrada inmediata para todos; progreso para quien lo quiera; no se almacenan contraseñas propias. (−) Dos tipos de identidad que el código debe contemplar (`profileId` puede ser nulo).

ADR-08 — Motores de juego enchufables

Contexto. La plataforma pasa de uno a tres minijuegos y debe admitir más. Se evaluaron dos juegos muy distintos entre sí (*Dibujá y Adiviná* y *Teléfono Descompuesto*).

Alternativas. (a) Lógica de cada juego dentro de los gateways con condicionales por tipo de juego; (b) **interfaz `GameEngine` + registro** (patrones *Strategy* y *Registry*); y, para elegir los juegos, (c) juegos nuevos con mecánicas propias o (d) **variantes de la mecánica de deducción**.

Decisión. (b) y (d). El resto del sistema conoce sólo la interfaz; la configuración de cada juego se describe con un `settingsSchema` que el lobby convierte en controles. Se eligió construir **Impostor por tiempo** e **Impostor dibujo** porque reutilizan toda la deducción (roles, discusión, votación, desempates, puntaje, pausa) a través de `DeductionEngine`: cada juego nuevo sólo define su actividad de la ronda. Eso permitió terminar tres juegos completos, probados y consistentes entre sí en el tiempo disponible, en lugar de dos juegos con reglas, pantallas y puntajes completamente distintos.

Consecuencias. (+) Un juego nuevo no toca salas, chat ni transporte (QS-7) y comparte las pantallas de discusión, votación y resultados; cada motor se prueba aislado. (−) Los tres juegos se parecen entre sí; un juego de otra familia (sin impostor) tendría que implementar `GameEngine` directamente. El dibujo necesitó un evento propio (`game:draw`) para los trazos en vivo, sin cambiar la interfaz.

ADR-09 — Vistas por jugador calculadas en el servidor

Contexto. Cada jugador debe ver una versión distinta del mismo estado (el impostor no ve la palabra; los votos se ocultan hasta el final).

Alternativas. (a) Enviar el estado completo y ocultar en la interfaz; (b) **calcular una vista por jugador en el servidor**.

Decisión. (b). Con (a) cualquier jugador podría leer la palabra en las herramientas del navegador (K2).

Consecuencias. (+) Integridad por diseño. (−) Un envío por socket en lugar de un único *broadcast* por sala; el costo es despreciable para salas de hasta 12 jugadores.

ADR-10 — Modo voz sin audio propio

Contexto. Muchos grupos juegan en persona o ya están en una llamada.

Alternativas. Integrar voz con WebRTC/LiveKit; **coordinar turnos y dejar el audio a herramientas externas**.

Decisión. Coordinar turnos solamente. Integrar audio multiplica la complejidad (servidores TURN, calidad de red, permisos de micrófono) y el costo, y no aporta a quien ya está en Discord o en la misma habitación.

Consecuencias. (+) Modo voz simple y gratuito. (−) En modo voz no hay registro de lo que se dijo; el sistema confía en los jugadores y el anfitrión puede avanzar turnos manualmente.

ADR-11 — Zustand para el estado del cliente

Contexto. El cliente necesita compartir pocos datos globales: identidad local, sala, chat y sesión.

Alternativas. Redux Toolkit; Context API; Zustand.

Decisión. Zustand: stores pequeños sin *providers*, mínimo código repetitivo y buena compatibilidad con componentes de servidor de Next.js. Redux resulta excesivo para tres stores; Context re-renderiza todo el árbol ante cada mensaje de chat.

ADR-12 — Monorepo con tipos compartidos

Contexto. Cliente y servidor deben coincidir exactamente en nombres y formas de decenas de eventos.

Alternativas. Repositorios separados con tipos duplicados; especificación OpenAPI/AsyncAPI con generación de código; **monorepo con paquete `shared`**.

Decisión. Monorepo con Turborepo. Un cambio en un evento se hace en un único lugar y rompe la compilación del lado que no se actualizó (QS-10). Turborepo garantiza que `shared` se compile antes que las aplicaciones.

Consecuencias. (+) Un solo pull request para cambios de punta a punta. (−) Build algo más complejo y un único repositorio para todo el equipo.

ADR-13 — Despliegue en Render

Contexto. RT-2 exige procesos de larga duración para WebSockets; RT-4 exige bajo costo.

Alternativa	Evaluación
Vercel (web) + Railway (server)	Vercel no admite WebSockets persistentes para el servidor; obligaba a dos proveedores.
Railway (web + server)	Un proveedor, deploy desde GitHub, procesos persistentes. No tiene plan gratuito permanente: la prueba gratuita vence y pasa a ser pago.
Render (web + server)	Plan gratuito con procesos persistentes y WebSockets, deploy automático desde `main` y configuración versionable (`render.yaml`). Los servicios gratuitos se duermen tras 15 minutos sin tráfico.
VPS propio	Máximo control y costo fijo bajo, pero exige administrar sistema operativo, TLS y actualizaciones (K7).

Tabla 28 — ADR-13: alternativas de despliegue

Decisión. Render para la web y el servidor; Supabase para Auth y PostgreSQL. El costo de que el servidor gratuito se duerma se absorbió en el código (§7 y §8.8) en lugar de pagar un plan; pasar al plan pago es un cambio de configuración, sin tocar código.

ADR-14 — Listado de salas públicas por REST

Contexto. El explorador de salas públicas muestra una lista que cambia, pero no requiere precisión al instante.

Alternativas. Suscripción por Socket.io a un canal global; **endpoint REST consultado cada 5 segundos**.

Decisión. REST (`GET /rooms/public`, consultado cada 5 segundos desde la página de inicio). La lista tolera 5 segundos de retraso; no obliga a abrir un socket antes de que el usuario elija una sala; y un canal global difundiría cada cambio de cada sala a todos los que están mirando la lista, un costo que crece con el éxito del producto.

ADR-15 — Resultados escritos sólo por el servidor

Contexto. Las estadísticas, los likes, las amistades y los reportes deben ser confiables.

Decisión. Los clientes sólo **leen** (RLS). Todas las escrituras que importan las hace el Game Server, único que conoce el desenlace real de una partida, con la clave secreta: las estadísticas, con la función `record_game_stats`; el workshop, las amistades y los reportes, después de validar cada pedido.

Consecuencias. (+) Nadie puede "inflar" sus estadísticas, likes o reportes desde el navegador. (−) El servidor necesita credenciales privilegiadas que deben protegerse (§8.4), y sin la clave el servidor funciona sólo para invitados.

ADR-16 — Amigos y workshop por el socket del juego

Contexto. Los amigos necesitan presencia en vivo ("en una sala", "jugando") e invitaciones instantáneas; el workshop, escrituras validadas.

Alternativas. (a) Supabase Realtime (suscripciones a cambios de la base) y escrituras directas desde el navegador con RLS; (b) **eventos por el mismo socket del servidor de juego**.

Decisión. (b). El servidor ya sabe quién está conectado y en qué sala, así que la presencia sale gratis de su memoria; no hace falta escribir en la base cada cambio de estado. Las escrituras pasan por el servidor, que valida (por ejemplo, 10 a 300 palabras por lista) y aplica el filtro de insultos; las **lecturas** del workshop van directo a Supabase, que las filtra con RLS.

Consecuencias. (+) Una sola conexión; reglas en un solo lugar; presencia sin costo de base. (−) Las solicitudes se guardan en la base, pero las invitaciones son sólo en vivo (vencen a los 5 minutos).

ADR-17 — Moderación: filtro compartido, reportes y panel de admin

Contexto. Con salas públicas y contenido creado por usuarios aparece el riesgo de abuso (R-4).

Alternativas. (a) Sólo expulsar desde la sala; (b) moderación automática con un servicio externo; (c) **filtro de insultos propio + reportes + panel de administración**.

Decisión. (c). El filtro vive en el paquete compartido (lo usan la web, para avisar antes, y el servidor, que decide): censura con `***` el chat y las pistas, y rechaza apodos, usuarios y contenido del workshop con insultos (contempla mayúsculas, acentos, letras estiradas y *leetspeak* sin marcar palabras inocentes como "computadora"). Los reportes adjuntan el chat como evidencia y los admins —una marca en el perfil— los resuelven desde `/admin`.

Consecuencias. (+) Sin costo ni dependencias externas; probado con casos borde. (−) Un filtro por lista nunca es perfecto: por eso existen los reportes.

ADR-18 — Internacionalización propia en lugar de una librería

Contexto. La interfaz está en dos idiomas y los textos del juego (fases, errores del servidor) cambian en vivo.

Alternativas. `next-intl` (rutas por idioma, mensajes por archivo); **diccionario propio tipado** (`lib/i18n.ts`) con una función `t(clave, idioma, parámetros)`.

Decisión. Diccionario propio. Todo el juego vive en una sola página de cliente (`/room/[code]`): rutas por idioma no aportan y obligarían a recargar al cambiar de idioma en medio de una partida. El servidor nunca manda texto, sólo **códigos** de error, que el cliente traduce (§8.3).

Consecuencias. (+) Cambio de idioma instantáneo sin recargar; sin dependencias. (−) Sin pluralización avanzada (no hizo falta con dos idiomas).

ADR-19 — Sonido sintetizado con Web Audio

Contexto. Se pidieron efectos y música, con costo y peso mínimos.

Alternativas. Archivos de audio (mp3/ogg) servidos con la web; **síntesis en el navegador con Web Audio**.

Decisión. Síntesis: cada efecto es una secuencia corta de notas (por ejemplo, la "campanita que sube" de tu turno) y la música son acordes y un arpegio suaves generados en el momento.

Consecuencias. (+) Cero bytes de audio que descargar, sin licencias, volumen y encendido independientes. (−) Sonidos más simples que una grabación; los navegadores exigen una interacción previa para reproducir audio (se desbloquea al primer toque).

ADR-20 — Identidad de invitado con un token por pestaña

Contexto. Para reconectar (RF-17) el servidor tiene que reconocer al mismo invitado después de un corte o una recarga, sin cuenta.

Alternativas. (a) Identificar por el id del socket (cambia en cada conexión); (b) cookie compartida por todo el navegador; (c) **token aleatorio en `sessionStorage`**, uno por pestaña.

Decisión. (c). Sobrevive a la recarga, y cada pestaña es un jugador distinto (útil para jugar o probar con varias pestañas). El token nunca se difunde a otros jugadores: el `playerId` público lo genera el servidor, así nadie puede hacerse pasar por otro copiando un id visible.

Consecuencias. (+) Reconexión transparente para invitados. (−) Cerrar la pestaña pierde la identidad de invitado (con cuenta no pasa: la identidad es la cuenta).

ADR-21 — Cuentas con email y contraseña (sin login con Google)

Contexto. Se evaluó ofrecer "Continuar con Google" además de email y contraseña.

Decisión. Sólo email y contraseña, con recuperación por mail. El login con Google exige registrar la aplicación en Google Cloud, una pantalla de consentimiento y dominios verificados; para una cuenta que es **opcional** (ADR-07) no compensaba ese trabajo. Supabase Auth permite agregarlo después sin cambiar el servidor, que sólo verifica el JWT.

Consecuencias. (+) Menos configuración y dependencias externas. (−) Un paso más para quien prefiere entrar con Google.

ADR-22 — Pruebas de punta a punta con navegador real y bots

Contexto. Muchos errores sólo aparecen con varios jugadores conectados a la vez (turnos, reconexión, mensajes que llegan mientras se navega).

Alternativas. (a) Sólo pruebas unitarias; (b) pruebas de punta a punta con 4 navegadores; (c) **un navegador real + bots que se conectan por Socket.io**.

Decisión. (c) en el CI y (b) para verificar producción. Un bot cuesta mucho menos que un navegador y controla con precisión qué hace cada jugador (votar a quién, cuándo desconectarse); el navegador verifica lo que ve el usuario. Las pruebas con cuentas crean usuarios temporales y los borran.

Consecuencias. (+) 17 suites que encontraron bugs reales (por ejemplo, el #29: mensajes perdidos al entrar a una sala). (−) Más lentas que las unitarias (unos 7 minutos en el CI).

ADR-23 — Mantener despierto el servidor gratuito en lugar de pagar

Contexto. Render gratis duerme el servidor a los 15 minutos sin tráfico HTTP y las salas en memoria se pierden (R-1).

Alternativas. (a) Plan pago (no duerme); (b) un servicio externo que haga ping siempre; (c) **ping desde los clientes que están en una sala + avisos claros**.

Decisión. (c). Mientras alguien juega, el servidor no se duerme (`GET /health` cada 4 minutos); cuando nadie juega, dormirse no le hace daño a nadie y no consume horas del plan. Si igual se pierde una sala, se avisa y se espera al servidor al crear o unirse.

Consecuencias. (+) Costo cero sin partidas cortadas por el sueño del servidor. (−) La primera conexión del día tarda hasta un minuto; pasar al plan pago lo resuelve sin tocar código.

10. Requerimientos de calidad

10.1 Árbol de calidad

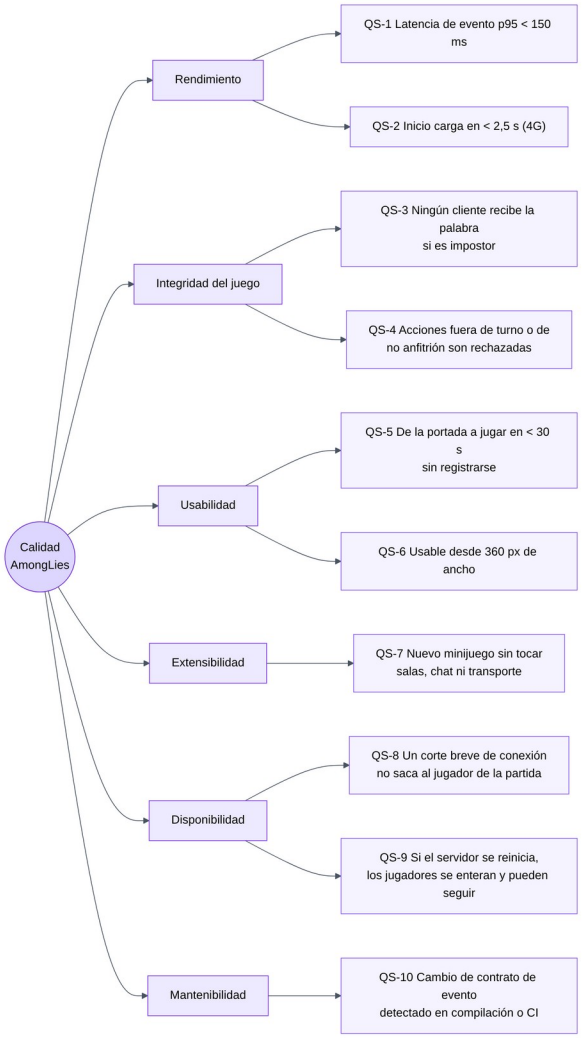


Figura 15 — Árbol de calidad con los escenarios que lo concretan

10.2 Escenarios de calidad

Cada escenario describe un estímulo, la respuesta esperada y una medida verificable.

ID	Atributo	Estímulo	Respuesta esperada	Medida / resultado
QS-1	Rendimiento	Un jugador vota en una sala de 8 jugadores.	Los 8 ven el contador actualizado.	p95 < 150 ms en la misma región.
QS-2	Rendimiento	Un usuario abre la página de inicio con 4G.	La página es interactiva.	LCP < 2,5 s (con el servidor despierto).
QS-3	Integridad	El impostor inspecciona el tráfico y el estado de la aplicación.	No encuentra la palabra ni el tiempo objetivo.	Verificado en tests unitarios de los tres motores.
QS-4	Integridad	Un jugador vota dos veces, juega fuera de turno o envía `game:start` sin ser anfitrión.	El servidor rechaza la acción.	100 % de rechazos en la suite e2e `servidor` (15 checks).
QS-5	Usabilidad	Una persona recibe un link de invitación por primera vez.	Está dentro del lobby.	< 30 s, sin registrarse.

ID	Atributo	Estímulo	Respuesta esperada	Medida / resultado
QS-6	Usabilidad	Se juega una partida completa en un celular de 360 px de ancho.	Todas las acciones son accesibles sin zoom.	0 páginas con scroll horizontal; e2e `dibujo-celular` y `chat` en celular.
QS-7	Extensibilidad	Se agrega un minijuego nuevo.	No se modifican salas, chat ni transporte.	Cumplido dos veces: Tiempo y Dibujo.
QS-8	Disponibilidad	A un jugador se le corta la conexión o recarga la página en medio de la partida.	Vuelve a su lugar; la partida lo espera pausada.	e2e `recarga` y `reconexion` (23 checks).
QS-9	Disponibilidad	El servidor se reinicia o se dormía.	Los jugadores se enteran y pueden seguir; crear una sala espera a que despierte.	e2e `sala-perdida`, que apaga y prende el servidor (6 checks).
QS-10	Mantenibilidad	Un desarrollador cambia la forma de un evento sólo en el servidor.	El error se detecta antes de desplegar.	Falla la compilación o el CI.

Tabla 29 — Escenarios de calidad

10.3 Trazabilidad: requerimiento → implementación → prueba

Cada requerimiento funcional se puede seguir hasta el código que lo implementa y la prueba que lo verifica:

RF	Dónde se implementa	Cómo se verifica	Ventana
RF-01–03	`app/page.tsx`, `room/room.gateway.ts`, `room.validation.ts`	Unit: validación de apodo/avatar. E2E: `chat`, `salas-publicas`.	V-05, V-06, V-12
RF-04	`lobby/Chat.tsx`, `chat/chat.gateway.ts`, filtro de insultos (shared)	Unit: `profanity.spec`. E2E: `chat`, `carrera-al-entrar`, `moderacion`.	V-13, V-19
RF-05	`GameSettings.tsx`, `game/settings.ts`	Unit: `settings.spec` (rangos, tipos, claves extra). E2E: `servidor`.	V-15
RF-06	`PlayerList.tsx`, `room.gateway.ts`, `shared/utils/decider.ts`	Unit: `decider.spec`. E2E: `reconexion`, `expulsado`.	V-13, V-20
RF-07	`ImpostorEngine`, `DeductionEngine`, `components/deduction`	Unit: motor del Impostor, votos, reglas, puntaje. E2E: `impostor`, `cuenta-y-estadisticas`.	V-21–V-27
RF-08	`TurnsVoice.tsx`, `advance` en el motor	Unit: `advance` sólo del anfitrión.	V-28
RF-09	`lib/i18n.ts`, listas por idioma	E2E: `chat` (interfaz en inglés), `salas-publicas` (idioma de la sala).	V-02
RF-10	`TimeEngine`, `time.rules.ts`, `components/time`	Unit: `time.rules`, `time.engine`. E2E: `tiempo`.	V-29–V-31
RF-11	`DrawingEngine`, `ink.ts`, `components/drawing`	Unit: `ink`, `drawing.engine`. E2E: `dibujo`, `dibujo-celular`.	V-32–V-34
RF-12–13	`RoomSettingsCard.tsx`, `PublicRooms.tsx`, `rooms.controller.ts`	Unit: `room.store.spec`. E2E: `salas-publicas`.	V-07, V-13

RF	Dónde se implementa	Cómo se verifica	Ventana
RF-14–15	`app/login`, `registro`, `recuperar`, `auth/token-verifier.ts`	Unit: `token-verifier.spec`. E2E: todas las suites con cuentas.	V-08–V-11
RF-16	`stats.service.ts`, `record_game_stats`, `app/perfil`	E2E: `cuenta-y-estadisticas` (lee la fila guardada).	V-35
RF-17	`PlayerService`, `PausableTimers`, `PauseOverlay.tsx`	Unit: `timers.spec`. E2E: `recarga`, `reconexion`, `sala-perdida`.	V-04, V-20
RF-18	`social/*`, `app/amigos`, `InviteFriends.tsx`	Unit: `social.rules`, `social.service`. E2E: `amigos`.	V-03, V-16, V-36
RF-19	`workshop/*`, `app/workshop`, `WorkshopLobbyTools.tsx`	Unit: `workshop.rules`, `workshop.service`. E2E: `workshop`.	V-16, V-37–V-39
RF-20	`moderation/*`, `ReportDialog.tsx`, `app/admin`	Unit: `moderation.rules`, `profanity`. E2E: `moderacion` (incluye RLS).	V-17, V-41–V-43
RF-21	`app/ajustes`, `settingsStore`, `lib/sound.ts`	E2E: `ajustes` (incluye borrar la cuenta).	V-40

Tabla 30 — Trazabilidad de los requerimientos funcionales

11. Riesgos y deuda técnica

Esta sección registra con honestidad los riesgos y la deuda técnica conocidos del sistema terminado.

11.1 Riesgos

ID	Riesgo	Prob.	Impacto	Mitigación (estado)
R-1	Un reinicio del servidor (deploy o plan gratis que lo duerme) pierde las salas activas.	Media	Alto	Keep-alive mientras hay jugadores, aviso de sala perdida y espera al despertar (hecho). Plan pago de Render para eliminarlo.
R-2	Una desconexión breve saca al jugador de la partida.	Alta	Alto	Reconexión con pausa de la partida (hecho, RF-17).
R-3	Límites de los planes gratuitos ante un pico de uso.	Media	Medio	Una instancia alcanza para el uso actual; salas con cupo; plan pago si crece.
R-4	Abuso en salas públicas y en el workshop (spam, insultos).	Media	Medio	Salas privadas por defecto, filtro de insultos, reportes, suspensiones y panel de admin (hecho).
R-5	Trampas fuera del sistema (compartir pantalla, contarse la palabra).	Media	Bajo	No mitigable técnicamente; es un juego entre amigos.
R-6	Supabase pausa los proyectos gratuitos tras 7 días sin actividad.	Alta	Medio	Pasó en octubre de 2026 y se restauró desde el panel. Mientras está pausado, el juego sigue para invitados; las cuentas no andan. Plan pago o actividad periódica.
R-7	Trazos del dibujo muy frecuentes saturan la conexión.	Baja	Medio	Puntos agrupados por mensaje, límites de puntos por mensaje y por turno, tinta limitada (hecho).
R-8	Dependencias con vulnerabilidades nuevas.	Media	Alto	`npm audit` en el CI frena el deploy (en octubre de 2026 detectó una crítica de Next.js, que se actualizó).

Tabla 31 — Riesgos

11.2 Deuda técnica

ID	Deuda	Decisión
TD-1	El estado vivo de las salas no sobrevive a un reinicio del servidor.	Aceptada (ADR-05); mitigada con keep-alive y aviso de sala perdida (R-1).
TD-2	No hay historial partida por partida en el perfil, sólo estadísticas acumuladas.	Pendiente: requeriría guardar cada partida en la base.
TD-3	Una sola instancia del servidor: no escala horizontalmente.	Aceptada (ADR-05); el camino para escalar está descrito en §8.9.

Tabla 32 — Deuda técnica

12. Glosario

Término	Definición
Sala	Espacio virtual donde un grupo de jugadores se reúne, chatea y juega. Se identifica con un código de 6 caracteres.
Código de sala	Identificador de 6 caracteres (letras y números sin ambigüedades como O/0 o I/1) que se comparte para invitar.
Anfitrión (host)	Jugador que administra la sala: elige juego, configura, inicia, expulsa. Es quien la crea, salvo que transfiera el rol.
Administrador (admin)	Usuario con permiso de moderación sobre toda la plataforma: revisa reportes y suspende cuentas. No confundir con el anfitrión de una sala.
Invitado	Usuario que juega sin cuenta, sólo con apodo y avatar.
Usuario registrado	Usuario con cuenta; conserva perfil, estadísticas, amigos y su colección del workshop.
Sala pública / privada	Una sala pública aparece en el listado de la página de inicio; una privada sólo es accesible con código, link o invitación.
Juego	Sesión completa de un minijuego desde "Iniciar" hasta la pantalla final.
Partida	Subdivisión del juego con los mismos impostores. Termina cuando un bando gana.
Ronda	Ciclo actividad → discusión → votación dentro de una partida.
Impostor / Inocente	Roles secretos: el impostor no conoce la palabra (o el tiempo objetivo); los inocentes sí.
Pista	Palabra que cada jugador da en su turno en El Impostor para demostrar que conoce la palabra secreta sin decirla.
Tiempo objetivo	En el Impostor por tiempo, la duración secreta que cada inocente intenta "parar" en su turno.
Tinta	En el Impostor dibujo, cuánto se puede dibujar por turno (100 % = una línea de lado a lado del lienzo).
Eliminado / espectador	Jugador expulsado por votación: sigue viendo la partida pero no puede chatear ni votar.
Modo chat / modo voz	En modo chat las pistas se escriben; en modo voz se dicen en voz alta y la app sólo controla los turnos.
Puntaje	Puntos por ronda (votar bien, sobrevivir) y por partida (ganar), que se muestran al terminar cada partida.
Workshop	Sección donde los usuarios crean, publican y guardan listas de palabras y presets de reglas.
Preset	Juego + todos sus ajustes, guardado para aplicarlo en el lobby con un clic.
Reporte / suspensión	Aviso a los admins sobre un jugador o contenido; una suspensión impide entrar a salas y publicar por un tiempo o para siempre.
Motor de juego (engine)	Componente del servidor que implementa las reglas de un minijuego como máquina de estados.
Vista de jugador	Versión del estado del juego filtrada para un jugador concreto.
Servidor autoritativo	Arquitectura en la que sólo el servidor decide el estado; los clientes envían intenciones.
Evento	Mensaje con nombre (por ejemplo `room:join`) que viaja por la conexión en tiempo real.
RLS	Row Level Security: reglas de PostgreSQL que limitan qué filas puede leer o escribir cada usuario.

Término	Definición
ADR	Architecture Decision Record: registro de una decisión de arquitectura con su contexto y consecuencias.
Prueba de punta a punta (e2e)	Prueba que usa el sistema completo como un usuario: navegador real, servidor y base.

Tabla 33 — Glosario

Anexo A — Referencias

- Starke, G. y Hruschka, P. *arc42 — Template for documentation of software and system architectures*. https://arc42.org
- Brown, S. *The C4 model for visualising software architecture*. https://c4model.com
- Working Software. *Software Architecture Documentation — The Ultimate Guide*. https://www.workingsoftware.dev/software-architecture-documentation-the-ultimate-guide/
- freeCodeCamp. *System Architecture Documentation Best Practices and Tools*. https://www.freecodecamp.org/news/system-architecture-documentation-best-practices-and-tools/
- Nygard, M. *Documenting Architecture Decisions* (formato ADR). https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions
- Documentación oficial: Socket.io (https://socket.io/docs/v4/), NestJS (https://docs.nestjs.com), Next.js (https://nextjs.org/docs), Supabase (https://supabase.com/docs), Render (https://render.com/docs), Playwright (https://playwright.dev), Jest (https://jestjs.io), Mermaid (https://mermaid.js.org).
- Ley 25.326 de Protección de los Datos Personales, República Argentina.
- Repositorio del proyecto: https://github.com/JuanCruzPalacios/AmongLies (incluye `docs/TESTING.md` con el detalle de las pruebas y `docs/PROGRESO.md` con las fases del desarrollo).
- Aplicación en producción: https://amonglies-web.onrender.com